Design and Analysis of Algorithms

Unit 1 Solutions

Basic Questions

Q1. Analyze the importance of algorithms in solving computational problems and their impact on efficiency.

- An algorithm is a finite, step-by-step procedure for solving a problem. It is the foundation of all computational problem solving.
- Importance in computational problem solving:
  - Provides a clear, unambiguous method to reach a solution from input to output.
  - Ensures correctness: a well-designed algorithm always produces the right answer.
  - Reusability: the same algorithm can be applied to different instances of the same problem.
- Impact on efficiency:
  - Time efficiency: A better algorithm reduces the number of operations (e.g., Binary Search O(log n) vs Linear Search O(n)).
  - Space efficiency: Optimal algorithms minimize memory consumption.
  - Scalability: Efficient algorithms handle large inputs within acceptable time and space bounds.
- Example: Sorting 1 million elements with Bubble Sort (O(n^2)) takes billions of operations, while Merge Sort (O(n log n)) takes only ~20 million - a dramatic efficiency difference.
- Conclusion: Choosing the right algorithm directly determines whether a problem can be solved in milliseconds or hours on modern hardware.

Q2. Differentiate between best-case, worst-case, and average-case complexities with suitable examples.

- Best-Case Complexity: The minimum time required by an algorithm for any input of size n. It represents the most favorable input scenario.
- Worst-Case Complexity: The maximum time required for any input of size n. It gives the upper bound on execution time.
- Average-Case Complexity: The expected time for a random input of size n, averaged over all possible inputs.

Case	Definition	Linear Search Example	Binary Search Example
Best Case	Most favorable input	O(1) - element at first position	O(1) - element at mid
Worst Case	Most unfavorable input	O(n) - element at last or absent	O(log n) - element absent
Average Case	Expected over all inputs	O(n/2) = O(n)	O(log n)

- Note: Worst-case is most commonly used in algorithm analysis as it provides a guaranteed upper bound that is safe for all inputs.

Q3. Analyze asymptotic notations and explain any two notations in terms of algorithm growth.

- Asymptotic notation describes the behavior of an algorithm's time or space complexity as the input size n approaches infinity. It abstracts away machine-specific constants.

1. Big-O Notation - O(g(n)) [Upper Bound / Worst Case]

- Definition: f(n) = O(g(n)) if there exist positive constants c and n0 such that f(n) <= c * g(n) for all n >= n0.
- Meaning: The algorithm grows no faster than g(n). It gives the worst-case growth rate.
- Example: f(n) = 3n^2 + 5n + 2 = O(n^2), because for large n, 3n^2 dominates all lower-order terms.
- Usage: Most commonly used - tells us the maximum resources needed.

2. Big-Omega Notation - Omega(g(n)) [Lower Bound / Best Case]

- Definition: f(n) = Omega(g(n)) if there exist positive constants c and n0 such that f(n) >= c * g(n) for all n >= n0.
- Meaning: The algorithm takes at least g(n) time. It gives the best-case or lower bound.
- Example: f(n) = 3n^2 + 5n + 2 = Omega(n^2), since f(n) >= c * n^2 for some c > 0.
- Usage: Used to prove no algorithm can do better than a certain bound.

3. Theta Notation - Theta(g(n)) [Tight Bound / Average]

- Definition: f(n) = Theta(g(n)) if f(n) = O(g(n)) AND f(n) = Omega(g(n)) simultaneously.
- Example: Merge Sort is Theta(n log n) in all cases - best, worst, and average are all n log n.

Q4. Discuss the concept of recurrence relations and determine their role in analyzing recursive algorithms.

- A recurrence relation is a mathematical equation that expresses the time complexity T(n) of a recursive algorithm in terms of the time complexity for smaller input sizes.
- General form: T(n) = a * T(n/b) + f(n), where:
  - a = number of recursive subproblems
  - n/b = size of each subproblem
  - f(n) = cost of work done outside recursive calls (divide + combine step)
- Role in analyzing recursive algorithms:
  - Captures the recursive structure of the algorithm mathematically.
  - Allows derivation of closed-form time complexity using methods like substitution, recursion tree, or Master Theorem.
  - Helps compare different recursive algorithms systematically.
- Example: Merge Sort divides the array into two halves and merges them in O(n) time.
  Recurrence: T(n) = 2T(n/2) + n
  Solution using Master Theorem: T(n) = O(n log n)
- Example: Binary Search eliminates half the search space each time.
  Recurrence: T(n) = T(n/2) + 1 => T(n) = O(log n)

Q5. Analyze the significance of time complexity in algorithm design and performance evaluation.

- Time complexity measures the number of basic operations an algorithm performs as a function of input size n. It is independent of hardware speed.
- Significance in algorithm design:
  - Guides the selection of the best algorithm before implementation.
  - Reveals scalability: O(n log n) handles 10^6 elements; O(n^3) cannot.
  - Identifies bottlenecks in a solution that need optimization.
- Significance in performance evaluation:
  - Provides a machine-independent measure to compare two algorithms.
  - Determines feasibility: whether a solution is practically usable for real data sizes.
- Hierarchy (slowest to fastest growing): O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n) < O(n!)
- Real-world impact: Google search requires O(n log n) or better algorithms - an O(n^2) algorithm on billions of web pages would be unusable.

Q6. Classify different types of time complexities and compare their efficiency.

Complexity	Name	Example Algorithm	n=10	n=100	n=1000
O(1)	Constant	Array index access	1	1	1
O(log n)	Logarithmic	Binary Search	3	7	10
O(n)	Linear	Linear Search	10	100	1000
O(n log n)	Linearithmic	Merge Sort, Heap Sort	33	664	9966
O(n^2)	Quadratic	Bubble Sort, Selection Sort	100	10000	10^6
O(n^3)	Cubic	Matrix Multiplication (naive)	1000	10^6	10^9
O(2^n)	Exponential	Subset generation	1024	10^30	Infeasible
O(n!)	Factorial	Brute-force TSP	3.6M	Infeasible	Infeasible

- Practical rule: O(1) to O(n log n) are considered efficient. O(n^2) is acceptable for small n. O(2^n) and O(n!) are infeasible for large inputs.

Q7. Analyze and apply the Master Theorem to solve recurrence relations in algorithm analysis.

- Master Theorem provides a direct formula to solve recurrences of the form:

T(n) = a * T(n/b) + f(n) where a >= 1, b > 1, f(n) is asymptotically positive

- Three cases:
- Case 1: If f(n) = O(n^(log_b(a) - epsilon)) for some epsilon > 0
  => T(n) = Theta(n^(log_b(a)))
  [f(n) grows polynomially SLOWER than n^(log_b(a))]
- Case 2: If f(n) = Theta(n^(log_b(a)) * log^k(n)) for k >= 0
  => T(n) = Theta(n^(log_b(a)) * log^(k+1)(n))
  [f(n) and n^(log_b(a)) grow at the SAME rate]
- Case 3: If f(n) = Omega(n^(log_b(a) + epsilon)) for some epsilon > 0, AND regularity condition holds
  => T(n) = Theta(f(n))
  [f(n) grows polynomially FASTER than n^(log_b(a))]

Example Applications:

- Binary Search: T(n) = T(n/2) + 1 => a=1, b=2, f(n)=1
  n^(log_2(1)) = n^0 = 1. f(n) = Theta(1). Case 2 (k=0) => T(n) = O(log n)

- Merge Sort: T(n) = 2T(n/2) + n => a=2, b=2, f(n)=n
  n^(log_2(2)) = n^1 = n. f(n) = Theta(n). Case 2 (k=0) => T(n) = O(n log n)

Q8. Compare deterministic and non-deterministic algorithms with examples based on their performance.

Feature	Deterministic Algorithm	Non-Deterministic Algorithm
Definition	For a given input, always produces the same output following a fixed sequence of steps	Can make random or guessed choices; may produce different outputs on same input
Predictability	Completely predictable	Output may vary between runs
Time Complexity	Polynomial time (P class) for tractable problems	Polynomial on non-deterministic machine (NP class)
Examples	Binary Search, Merge Sort, Dijkstra's	Randomized QuickSort, Las Vegas algorithms
Performance	Guaranteed worst-case bound	Expected good performance; worst case may be poor
Machine model	Deterministic Turing Machine	Non-Deterministic Turing Machine

- Key concept: Deterministic = same path every time. Non-deterministic = can "guess" the best choice and verify in polynomial time.
- The P vs NP problem asks: can every non-deterministic polynomial algorithm be simulated by a deterministic polynomial algorithm?

Q9. Explain the relationship between P, NP, NP-Hard, and NP-Complete using a Venn diagram.

Definitions:

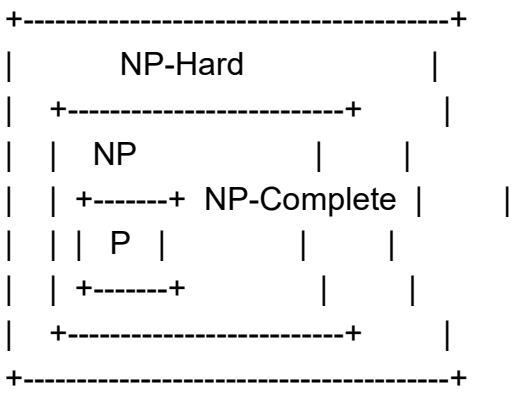
- P (Polynomial Time): Class of problems solvable by a deterministic algorithm in polynomial time O(n^k). Examples: Sorting, Binary Search, Shortest Path.
- NP (Non-deterministic Polynomial): Problems for which a given solution can be verified in polynomial time on a deterministic machine. Examples: SAT, Traveling Salesman (decision), Graph Coloring.
- NP-Hard: Problems at least as hard as the hardest problems in NP. Not necessarily in NP themselves. Example: Halting Problem, Optimization TSP.
- NP-Complete: Problems that are both in NP AND NP-Hard. Every NP problem reduces to an NP-Complete problem in polynomial time. Examples: SAT, Hamiltonian Cycle, Vertex Cover.

Venn Diagram relationship (assuming P != NP):

P subset of NP subset of NP-Hard

NP-Complete = NP intersection NP-Hard

Diagram (text representation):



- Key relationships: P is subset of NP. If any NP-Complete problem is solved in polynomial time, then P = NP. This is the greatest unsolved problem in computer science.

Moderate Questions

M1. Analyze the relationship between upper bound and lower bound in evaluating algorithm efficiency.

- Upper Bound (Big-O): The maximum time an algorithm can take. O(g(n)) means the algorithm never exceeds c * g(n) operations for large n.
- Lower Bound (Big-Omega): The minimum time any algorithm must take to solve a problem. Omega(g(n)) means no algorithm can do it faster than c * g(n) for large n.
- Tight Bound (Theta): When upper bound equals lower bound, the complexity is precisely determined.
- Relationship in evaluating efficiency:

  - If O(f(n)) = Omega(f(n)) for an algorithm, then T(n) = Theta(f(n)) - the algorithm is asymptotically optimal.

  - The gap between upper and lower bounds measures how much room there is for improvement.

  - Example: Comparison-based sorting has Omega(n log n) lower bound. Merge Sort achieves O(n log n) => it is optimal.
- Practical implication: If your algorithm matches the problem's lower bound, no fundamentally better algorithm exists.

M2. Classify asymptotic notations based on their functions in algorithm analysis.

- Asymptotic notations are mathematical tools that describe the growth rate of functions for large inputs. They are classified as:

Notation	Name	Meaning	Example
O(g(n))	Big-O	Upper bound - worst case growth rate	Bubble Sort = O(n^2)
Omega(g(n))	Big-Omega	Lower bound - best case growth rate	Merge Sort = Omega(n log n)
Theta(g(n))	Big-Theta	Tight bound - exact growth rate	Merge Sort = Theta(n log n)
o(g(n))	Little-o	Strict upper bound (not tight)	2n = o(n^2)
omega(g(n))	Little-omega	Strict lower bound (not tight)	n^2 = omega(n)

- In practice: Big-O is most commonly used as it gives the worst-case guarantee. Theta is used when exact analysis is needed. Omega proves impossibility results.

M3. Analyze the best-case, worst-case, and average-case time complexities of Linear Search.

- Linear Search: Scans each element of an array sequentially until the target is found or all elements are checked.

Algorithm:

```
for i = 0 to n-1:
    if A[i] == target: return i
return -1
```

Complexity Analysis:

- Best Case - O(1): Target element is at the first position. Only 1 comparison needed.
- Worst Case - O(n): Target is at the last position or absent. n comparisons needed.
- Average Case - O(n): Assuming the element is equally likely to be anywhere, average comparisons = (1+2+...+n)/n = (n+1)/2 = O(n).

Case	Condition	Operations	Time Complexity
Best	Target at index 0	1 comparison	O(1)
Worst	Target absent or at index n-1	n comparisons	O(n)
Average	Target equally likely anywhere	(n+1)/2 comparisons	O(n)

M4. Analyze the best-case, worst-case, and average-case time complexities of operations in a Binary Search Tree (BST).

Operation	Best Case	Average Case	Worst Case	Worst Case Condition
Search	O(1) - root is target	O(log n)	O(n)	Skewed (unbalanced) BST
Insert	O(1) - insert at root	O(log n)	O(n)	Inserting sorted sequence
Delete	O(1) - leaf deletion	O(log n)	O(n)	Skewed tree
Traversal	O(n)	O(n)	O(n)	Always visits all nodes

- Best case: Occurs in a balanced BST where height = log n. Operations take O(log n) or O(1).
- Worst case: Occurs when BST degenerates into a linked list (e.g., inserting 1, 2, 3, 4... in order creates a right-skewed tree). Height becomes n.
- Solution: Use self-balancing BSTs like AVL Tree or Red-Black Tree that maintain O(log n) height always.

M5. Explain the importance of recurrence relations in algorithm analysis using a suitable example.

- Recurrence relations are mathematical equations that define the time complexity of recursive algorithms in terms of smaller inputs. They are essential for:
- 1. Formally defining recursive complexity: Instead of counting operations manually, the recurrence captures the algorithm structure.
- 2. Enabling systematic solution: Master Theorem, Substitution, Recursion Tree methods solve recurrences systematically.
- 3. Comparing recursive algorithms: Different recurrences reveal which algorithm is faster.

Example - Merge Sort:

- Algorithm structure: Divide array into two halves T(n/2) each. Merge step takes O(n).
- Recurrence: T(n) = 2T(n/2) + n, with base case T(1) = 1
- Solution via Master Theorem: a=2, b=2, f(n)=n. n^(log_b a) = n^1 = n. f(n) = Theta(n) => Case 2 => T(n) = O(n log n)
- Without recurrence: We would need to manually trace through every recursive call for every input size, which is impractical.

M6. Analyze how the Master Theorem is applied to solve recurrence relations efficiently.

- The Master Theorem solves divide-and-conquer recurrences of the form T(n) = aT(n/b) + f(n) in three steps:
- Step 1: Identify a, b, and f(n) from the recurrence.
- Step 2: Compute the critical exponent p = log_b(a).
- Step 3: Compare f(n) with n^p:
  - If f(n) < n^p polynomially => Case 1: T(n) = Theta(n^p)
  - If f(n) = Theta(n^p * log^k n) => Case 2: T(n) = Theta(n^p * log^(k+1) n)
  - If f(n) > n^p polynomially => Case 3: T(n) = Theta(f(n))

Application Example: T(n) = 4T(n/2) + n

a=4, b=2, f(n)=n. p = log_2(4) = 2. n^p = n^2.
f(n) = n = O(n^(2-1)) => Case 1 applies.
Result: T(n) = Theta(n^2)

M7. Analyze and solve the recurrence relation using the Master Theorem: T(n) = 8T(n/2) + n^2

Given: T(n) = 8T(n/2) + n^2

Step 1: Identify parameters

a = 8 (number of subproblems)
b = 2 (subproblem size factor)
f(n) = n^2 (cost of non-recursive work)

Step 2: Compute critical exponent

log_b(a) = log_2(8) = log_2(2^3) = 3
n^(log_b a) = n^3

Step 3: Compare f(n) with n^(log_b a)

f(n) = n^2 vs n^(log_b a) = n^3

n^2 = O(n^(3 - 1)) => f(n) grows polynomially SLOWER than n^3

=> Case 1 of Master Theorem applies

Final Answer: T(n) = Theta(n^3) = O(n^3)

- Interpretation: This recurrence could represent an algorithm like naive matrix multiplication where the dominant cost is from the recursive structure (8 subproblems), not the combining step (n^2).

M8. Formulate the recurrence relation of Binary Search and solve it using the Master Theorem.

Binary Search Recurrence Formulation:

- Algorithm: Compare target with middle element. If equal, found. If target < mid, search left half. If target > mid, search right half.
- At each step: 1 comparison (O(1)) + 1 recursive call on n/2 elements.

Recurrence: T(n) = T(n/2) + 1, T(1) = 1

Solve using Master Theorem:

a = 1, b = 2, f(n) = 1 = O(1) = O(n^0)

log_b(a) = log_2(1) = 0

n^(log_b a) = n^0 = 1

f(n) = 1 = Theta(n^0 * log^0 n) = Theta(1)

=> Case 2 applies with k = 0

T(n) = Theta(n^0 * log^(0+1) n) = Theta(log n)

Final Answer: T(n) = O(log n)

- This matches the intuitive understanding: Binary Search halves the search space each step, taking log_2(n) steps total.

M9. Analyze and determine the upper bound, lower bound, and average bound for the function: f(n) = 2n^2 + 3n + 4

Given: f(n) = 2n^2 + 3n + 4

Upper Bound (Big-O):

- We need to find g(n) such that f(n) <= c * g(n) for all n >= n0.

  f(n) = 2n^2 + 3n + 4 <= 2n^2 + 3n^2 + 4n^2 = 9n^2 for all n >= 1

  With c = 9, n0 = 1:

f(n) <= 9n^2 for all n >= 1

=> f(n) = O(n^2)

Lower Bound (Big-Omega):

- We need to find g(n) such that f(n) >= c * g(n) for all n >= n0.
  f(n) = 2n^2 + 3n + 4 >= 2n^2 for all n >= 1
  With c = 2, n0 = 1:

=> f(n) = Omega(n^2)

Tight Bound (Theta):

Since f(n) = O(n^2) AND f(n) = Omega(n^2)

=> f(n) = Theta(n^2)

- Conclusion: The dominant term n^2 determines all three bounds. Lower-order terms (3n + 4) are absorbed asymptotically.

M11. Formulate the recurrence relation for Merge Sort.

- Merge Sort follows the Divide and Conquer strategy:
- 1. Divide: Split the array of n elements into two halves of n/2 elements each. Cost: O(1)
- 2. Conquer: Recursively sort both halves. Two recursive calls each on n/2 elements: 2 * T(n/2)
- 3. Combine (Merge): Merge two sorted halves into one sorted array. Cost: O(n)

Recurrence Relation:

T(n) = 2T(n/2) + n

T(1) = 1 (base case: array of 1 element is already sorted)

- Solving with Master Theorem: a=2, b=2, f(n)=n. log_2(2)=1. n^1 = n. f(n) = Theta(n) => Case 2 => T(n) = O(n log n)

M12a. Analyze and determine the upper bound for the function: f(n) = n^2 log n + n

Given: f(n) = n^2 log n + n

Finding Upper Bound O(g(n)):

f(n) = n^2 log n + n

For n >= 1: n <= n^2 log n (since log n >= 1 for n >= 2)

Therefore: f(n) = n^2 log n + n <= n^2 log n + n^2 log n = 2n^2 log n

With c = 2, n0 = 2:

f(n) = O(n^2 log n)

- The dominant term is n^2 log n. The term n is of lower order and does not affect the asymptotic upper bound.

M12b. Compute and justify the upper bound for the function: f(n) = 2n^3 + 4n + 5

Given: f(n) = 2n^3 + 4n + 5

Finding Upper Bound:

For n >= 1:

4n <= 4n^3 and 5 <= 5n^3

Therefore: f(n) = 2n^3 + 4n + 5 <= 2n^3 + 4n^3 + 5n^3 = 11n^3

With c = 11, n0 = 1:

f(n) = O(n^3)

- Justification: The highest-degree term n^3 dominates for large n. All lower-degree terms (4n, 5) become negligible. The constant factor 2 is absorbed into the constant c in the Big-O definition.

M13. Analyze and compute the time complexity of Merge Sort and explain its efficiency.

Merge Sort - Complete Time Complexity Analysis:

- Recurrence: T(n) = 2T(n/2) + n

Recursion Tree Analysis:

Level 0: n (cost n)

Level 1: n/2 + n/2 = n (cost n)

Level 2: 4 * n/4 = n (cost n)

...

Level log n: n nodes of size 1 (cost n)

Total levels = log n + 1

Total cost = n * (log n + 1) = n log n + n = Theta(n log n)

Case	Time Complexity	Explanation
Best Case	O(n log n)	Always divides equally, merge still takes O(n)
Worst Case	O(n log n)	No bad partitioning possible - always O(n log n)
Average Case	O(n log n)	Consistent performance regardless of input

Space Complexity: O(n)

- Efficiency: Merge Sort is optimal for comparison-based sorting. It is efficient for large datasets and linked lists. Disadvantage: requires O(n) extra space for merging.

M14. Solve the recurrence T(n) = 2T(n/2) + n and interpret its time complexity.
Given: T(n) = 2T(n/2) + n

Master Theorem Application:

a = 2, b = 2, f(n) = n
log_b(a) = log_2(2) = 1
n^(log_b a) = n^1 = n
f(n) = n = Theta(n^1) = Theta(n^(log_b a) * log^0 n)
=> Case 2 applies with k = 0
T(n) = Theta(n^1 * log^(0+1) n) = Theta(n log n)

Final Answer: T(n) = O(n log n)

- Interpretation: This recurrence describes Merge Sort. The algorithm makes 2 recursive calls on half-sized subproblems and does O(n) work at each level. The resulting O(n log n) is optimal for comparison-based sorting.

M15. Solve the recurrence T(n) = 4T(n/2) + n^2 using the Master Theorem.
Given: T(n) = 4T(n/2) + n^2

Step 1: Identify parameters

a = 4, b = 2, f(n) = n^2

Step 2: Compute critical exponent

log_b(a) = log_2(4) = 2
n^(log_b a) = n^2

Step 3: Compare f(n) with n^(log_b a)

f(n) = n^2 = Theta(n^2) = Theta(n^(log_b a) * log^0 n)

=> Case 2 applies with k = 0

T(n) = Theta(n^2 * log^(0+1) n) = Theta(n^2 log n)

Final Answer: T(n) = O(n^2 log n)

- This recurrence appears in algorithms like Strassen's matrix multiplication variant where 4 subproblems of size n/2 are solved with O(n^2) combining step.

M16. Determine the time complexity of T(n) = T(n/2) + 1 and relate it to a standard algorithm.

Given: T(n) = T(n/2) + 1

Master Theorem Application:

a = 1, b = 2, f(n) = 1 = O(1) = O(n^0)

log_b(a) = log_2(1) = 0

n^(log_b a) = n^0 = 1

f(n) = 1 = Theta(1) = Theta(n^0 * log^0 n)

=> Case 2 with k = 0

T(n) = Theta(log^(0+1) n) = Theta(log n)

Final Answer: T(n) = O(log n)

Related standard algorithm: Binary Search

- Binary Search makes one recursive call on half the input (T(n/2)) and does O(1) work for comparison (+1). This matches the recurrence exactly.
- The O(log n) complexity means Binary Search on 1 billion elements needs only ~30 comparisons.

M17. Analyze whether the Master Theorem is applicable to T(n) = 2T(n/2) + n log n and justify your answer.

Given: T(n) = 2T(n/2) + n log n

Step 1: Identify parameters

a = 2, b = 2, f(n) = n log n

Step 2: Compute critical exponent

log_b(a) = log_2(2) = 1

n^(log_b a) = n^1 = n

Step 3: Check Master Theorem cases

f(n) = n log n. Compare with n^1 = n.

f(n) = n log n vs n^1 = n

f(n) / n^(log_b a) = n log n / n = log n

log n is not O(1) => f(n) is NOT polynomially smaller than n^1 (Case 1 fails)

log n is not Omega(n^epsilon) for any epsilon > 0 => f(n) is NOT polynomially larger (Case 3 fails)

f(n) = n log n = Theta(n * log^1 n) = Theta(n^(log_b a) * log^1 n) => Case 2 applies with k = 1

Master Theorem IS applicable. Case 2 with k = 1:

T(n) = Theta(n^(log_b a) * log^(k+1) n) = Theta(n * log^2 n) = O(n log^2 n)

- This recurrence appears in merge sort variants with a non-linear merge step. The extra log factor makes it slightly slower than standard merge sort O(n log n).

Descriptive / HOTS Questions

HOTS Q1. Analyze the application of asymptotic notations in evaluating algorithm efficiency, and interpret their impact using multiple examples.

- Asymptotic notations are the primary tool for analyzing and comparing algorithm efficiency. They abstract away hardware differences and focus on the fundamental growth rate of algorithms.

Application 1 - Sorting Algorithms:

- Bubble Sort: O(n^2) in worst/average case. For n=10000 elements: ~10^8 operations. Impractical for large inputs.
- Merge Sort: O(n log n) always. For n=10000: ~132,877 operations. Dramatically more efficient.
- Impact: Asymptotic analysis reveals that for large n, O(n log n) algorithms are orders of magnitude faster than O(n^2).

Application 2 - Search Algorithms:

- Linear Search: O(n) - searches one by one. For 1 billion entries: up to 10^9 comparisons.
- Binary Search: O(log n) - halves space each step. For 1 billion entries: only 30 comparisons!

- Impact: The exponential difference in speed makes binary search essential for large-scale databases.

Application 3 - Graph Algorithms:

- Dijkstra with array: O(V^2). Dijkstra with min-heap: O((V+E) log V). For V=10^6: array version infeasible, heap version viable.

Interpretation of impact:

- O(1) vs O(log n): For n=10^9, difference is between 1 operation and 30 operations - negligible.
- O(n) vs O(n^2): For n=10^6, difference is between 10^6 and 10^12 operations - critical for feasibility.
- O(n log n) vs O(2^n): For n=50, O(n log n) ~280 operations; O(2^n) ~10^15 operations - completely impractical.
- Conclusion: Asymptotic notation is the foundation of algorithm engineering. Choosing the right algorithm can determine whether a computation takes milliseconds or millions of years.

HOTS Q2. Investigate the formulation of recurrence relations and apply the Master Theorem to derive their time complexity, supporting your answer with a detailed example.

- Recurrence relation formulation follows a systematic approach based on the recursive algorithm structure:

Formulation Steps:

- 1. Count recursive calls: How many times does the function call itself? => value of a
- 2. Determine subproblem size: By what factor is the input reduced? => value of b
- 3. Measure non-recursive work: What is the cost of divide + combine steps? => f(n)
- 4. Write the recurrence: T(n) = a * T(n/b) + f(n)

Detailed Example - Quick Sort (Average Case):

- Algorithm: Choose pivot. Partition array around pivot in O(n). Recursively sort both partitions.
- Average case: Pivot splits array into roughly equal halves.
- Recurrence: T(n) = 2T(n/2) + n

Master Theorem Solution:

a = 2, b = 2, f(n) = n

log_b(a) = log_2(2) = 1, so n^(log_b a) = n

f(n) = n = Theta(n^1 * log^0 n) => Case 2 with k=0

T(n) = Theta(n log n)

Worst Case Quick Sort (sorted input, poor pivot):

- Recurrence: T(n) = T(n-1) + n (one partition empty, one has n-1 elements)
- Solving: T(n) = n + (n-1) + ... + 1 = n(n+1)/2 = O(n^2)
- Conclusion: Recurrence formulation clearly reveals why Quick Sort degrades to O(n^2) for sorted inputs and how randomized pivot selection is essential for guaranteed O(n log n) average performance.

HOTS Q3. Formulate the recurrence relation for multiplication of large digit number and derive its time complexity using the Master Theorem, justifying each step.

- Traditional Multiplication: Multiplying two n-digit numbers naively requires O(n^2) operations. Karatsuba's algorithm improves this using divide and conquer.

Karatsuba Algorithm Formulation:

- Split each n-digit number into two n/2-digit halves:

  X = a * 10^(n/2) + b (a = upper half, b = lower half)

  Y = c * 10^(n/2) + d (c = upper half, d = lower half)

  X * Y = ac * 10^n + (ad + bc) * 10^(n/2) + bd

Key insight: Instead of 4 multiplications (ac, ad, bc, bd), compute only 3:

m1 = a * c

m2 = b * d

m3 = (a + b) * (c + d) = ac + ad + bc + bd

ad + bc = m3 - m1 - m2 (no extra multiplication!)

Recurrence Relation:

T(n) = 3T(n/2) + O(n)

where: 3 recursive multiplications + O(n) for additions and shifts

Master Theorem Derivation:

a = 3, b = 2, f(n) = O(n)

Step 1: log_b(a) = log_2(3) = 1.585

Step 2: n^(log_b a) = n^1.585

Step 3: f(n) = n = O(n^1) vs n^1.585

Since 1 < 1.585: f(n) = O(n^(1.585 - 0.585)) => Case 1 applies

T(n) = Theta(n^(log_2 3)) = Theta(n^1.585)

Final Answer: T(n) = O(n^1.585)

- Justification: Traditional multiplication is O(n^2). Karatsuba reduces this to O(n^1.585) by eliminating one multiplication per level using algebraic manipulation. For large numbers (hundreds of digits), this is a substantial improvement - the algorithm is used in practice for big integer arithmetic in cryptography.

HOTS Q4. Compare and evaluate the time complexities of Merge Sort and Quick Sort across best, worst, and average cases, and infer their suitability for different scenarios.

Case	Merge Sort	Quick Sort (Random Pivot)	Quick Sort (Poor Pivot)
Best Case	O(n log n)	O(n log n)	O(n^2) [if always picking min/max]
Average Case	O(n log n)	O(n log n)	O(n log n) expected
Worst Case	O(n log n)	O(n log n) expected	O(n^2) [sorted/reverse-sorted input]
Space Complexity	O(n) extra (merge)	O(log n) stack (average)	O(n) stack (worst)
Stability	Stable	Not stable	Not stable
In-place	No	Yes	Yes

Detailed Evaluation:

- Merge Sort: Guarantees O(n log n) in ALL cases. Predictable performance is its key strength. The O(n) extra space is a tradeoff. Excellent for linked lists and external sorting (sorting data on disk).
- Quick Sort with Random Pivot: O(n log n) expected with very small constants in practice. Performs better than Merge Sort on cache due to in-place operation. Risk of O(n^2) worst case is negligible with randomization.

Suitability inference:

- Use Merge Sort when:
  - Stable sort is required (preserving relative order of equal elements)
  - Worst-case guarantee of O(n log n) is mandatory (e.g., real-time systems)
  - Sorting linked lists (no random access needed)
  - External sorting (data does not fit in RAM)

- Use Quick Sort when:
  - In-place sorting is required (limited memory)
  - Average-case performance matters more than worst-case guarantee
  - Sorting random or unpredictable data (cache-friendly, faster in practice)
- Conclusion: Quick Sort is generally faster in practice due to lower constants and cache efficiency. Merge Sort is preferred when correctness guarantees and stability matter more than raw speed.

HOTS Q5. Critically evaluate a real-world scenario where best-case analysis is more significant than worst-case analysis, and justify your reasoning based on algorithm performance.

- Common wisdom in algorithm analysis prioritizes worst-case analysis. However, there are important real-world scenarios where best-case or average-case analysis is more meaningful.

Scenario: Real-Time Search Suggestion (Auto-complete in Search Engines)

- Problem: As a user types each character, the system searches millions of entries to suggest completions. Response must appear within 50ms for good user experience.
- Algorithm: Linear search through indexed results, with early termination.

Why best-case analysis matters here:

- 1. Data is pre-sorted and indexed: Popular queries appear first. In 80% of cases, the top-k results are found in the first portion of the index => Best case is common, not rare.
- 2. Early termination design: The algorithm is explicitly designed to return after finding k results. Best-case behavior is the expected behavior.
- 3. User experience threshold: Users perceive responses under 100ms as instant. If best-case is O(1) and worst-case is O(n), designing for worst-case would lead to over-engineering that never benefits typical users.

Additional real-world scenarios:

- Insertion Sort for nearly-sorted data: Worst case O(n^2), but best case O(n) for already-sorted input. In databases where records are incrementally inserted in order, nearly-sorted data is the norm, making best-case dominant.
- Hash table lookup: Worst case O(n) with collisions, best/average case O(1). In practice, good hash functions make O(1) the routine case, making worst-case analysis misleading.

Justification:

- Worst-case analysis is essential for safety-critical systems (medical, aviation). But for user-facing applications and systems with controlled data distributions, average-case or best-case analysis aligns better with actual system performance and guides better design decisions.
- Conclusion: Algorithm analysis should always be contextualized. The "right" analysis depends on the distribution of real inputs the algorithm will actually process, not just theoretical worst-case inputs.

Unit 2 Solutions

Q1. How does binary search utilize the divide and conquer strategy?

Answer:

Problem Understanding:

- Binary Search finds a target element in a sorted array by repeatedly halving the search space.
- It perfectly exemplifies all three phases of Divide and Conquer.

Divide and Conquer Phases:

- Divide: Compare target with the middle element of the current range.
- Conquer: If target < mid, recurse on left half. If target > mid, recurse on right half.
- Combine: No combining needed. The result is returned directly from the recursive call.

Algorithm:

```
BinarySearch(A, low, high, target):
  if low > high: return -1          // Not found
  mid = (low + high) / 2
  if A[mid] == target: return mid   // Found
  if target < A[mid]:  return BinarySearch(A, low, mid-1,
target)
  else:                return BinarySearch(A, mid+1, high,
target)
```

Time and Space Complexity:

- Recurrence: T(n) = T(n/2) + O(1)
- By Master Theorem (Case 2): T(n) = O(log n)
- Space Complexity: O(log n) for recursive call stack; O(1) for iterative version.

Q2. Differentiate between quick sort and merge sort in terms of efficiency.

Answer:

Criteria	Quick Sort	Merge Sort
Strategy	Partition around pivot, recurse	Split in half, recurse, merge
Best Case	O(n log n) – balanced pivot	O(n log n) – always
Average Case	O(n log n)	O(n log n)
Worst Case	O(n2) – already sorted, bad pivot	O(n log n) – always
Space	O(log n) – in-place (stack)	O(n) – requires extra array
Stability	Not stable	Stable

Cache Efficiency	Excellent – in-place access	Poor – accesses non-sequential memory
Preferred for	Arrays, in-memory data	Linked lists, external sorting

Conclusion: Quick Sort is faster in practice due to better cache performance, but Merge Sort guarantees O(n log n) in all cases.

Q3. Explain how the divide and conquer strategy applies to multiplication of large digit numbers.

Answer:

Problem Understanding:

- Multiplying two n-digit numbers naively takes O(n2) operations (grade school multiplication).
- The Karatsuba algorithm uses Divide and Conquer to reduce this to O(n^1.585).

Karatsuba's Divide and Conquer Idea:

- Split each n-digit number X and Y into two halves of n/2 digits each:

X = X_H * 10^(n/2) + X_L Y = Y_H * 10^(n/2) + Y_L

- Naive D&C uses 4 multiplications: X_H*Y_H, X_H*Y_L, X_L*Y_H, X_L*Y_L
- Karatsuba reduces to only 3 multiplications:

m1 = X_H * Y_H

m2 = X_L * Y_L

m3 = (X_H + X_L) * (Y_H + Y_L)

Cross-term = m3 - m1 - m2 (equals X_H*Y_L + X_L*Y_H)

Final result = m1*10^n + (cross-term)*10^(n/2) + m2

Recurrence and Complexity:

- Recurrence: T(n) = 3T(n/2) + O(n)
- By Master Theorem Case 1: T(n) = O(n^log2(3)) = O(n^1.585)
- This is significantly better than naive O(n2) for large numbers.

Q4. Write the control abstraction of the greedy strategy and analyse its time complexity.

Answer:

Control Abstraction:

```
Greedy(Input_Set S):
  Solution = {}              // Initialize empty solution
  while S is not empty:
    x = Select_Best(S)       // Greedy choice – pick most
promising element
    S = S - {x}              // Remove selected element
    if Feasible(Solution ∪ {x}):
```

```
        Solution = Solution ∪ {x}    // Add to solution if
feasible
  return Solution
```

Key Properties Required for Greedy to Work:

- Greedy Choice Property: A globally optimal solution can be built by making locally optimal (greedy) choices.
- Optimal Substructure: Optimal solution to problem contains optimal solutions to its subproblems.

Time Complexity Analysis:

- Select_Best() step: O(n) per iteration if scanning linearly, or O(log n) using a max-priority queue.
- Total iterations: O(n) – one per element.
- Overall Time Complexity: O(n log n) – dominated by sorting or priority queue operations.
- Space Complexity: O(n) for storing the solution set.

Q5. Discuss the knapsack problem, and how is it solved using the greedy method?

Answer:

Problem Understanding:

- Knapsack Problem: Given n items with weights w[i] and values v[i], and a knapsack of capacity W, maximize total value without exceeding weight limit.
- Two variants: Fractional Knapsack (items can be split – greedy works optimally) and 0/1 Knapsack (no splitting – greedy may fail).

Greedy Approach for Fractional Knapsack:

- Step 1: Compute value-to-weight ratio v[i]/w[i] for each item.
- Step 2: Sort items in decreasing order of ratio.
- Step 3: Greedily pick items from highest ratio to lowest. Take full item if it fits; take fractional part of last item if needed.

Example:

Items: (w=2,v=10), (w=3,v=5), (w=5,v=15). Capacity W=7.

Ratios: 5.0, 1.67, 3.0. Sorted: Item1(5.0), Item3(3.0), Item2(1.67).

Take Item1 fully (w=2), Take Item3 fully (w=5). Total weight=7=W.

Final Answer: Total Value = 10 + 15 = 25

Time Complexity:

- O(n log n) for sorting + O(n) for selection = O(n log n) overall.

Q6. How does Dijkstra's algorithm find the shortest path in a graph?

Answer:

Problem Understanding:

- Dijkstra's algorithm finds shortest paths from a single source vertex s to all other vertices in a weighted graph with non-negative edge weights.
- It uses a Greedy strategy – always processes the unvisited vertex with the smallest known distance.

Algorithm Steps:

- Step 1: Initialize dist[s] = 0, dist[v] = infinity for all other vertices. Use a min-priority queue.
- Step 2: Extract vertex u with minimum distance from the priority queue.
- Step 3: For each neighbor v of u: if dist[u] + w(u,v) < dist[v], update dist[v] = dist[u] + w(u,v) (relaxation).
- Step 4: Repeat Steps 2-3 until all vertices are processed.

Why It Works (Greedy Proof):

- With non-negative weights, once a vertex is extracted from the priority queue, its shortest distance is finalized.
- No negative edge can provide a shorter path after the vertex is settled.

Time Complexity:

- With binary heap (min-priority queue): O((V + E) log V)
- With adjacency matrix (simple array): O(V2)
- Space Complexity: O(V) for distance array and priority queue.

Moderate Questions

Q7. Apply the quick sort algorithm to the following array: [7, 2, 9, 4, 3, 1, 5], and show the steps.

Answer:

Problem Understanding:

- Quick Sort selects a pivot element, partitions the array so that all elements less than pivot come before it and greater elements come after, then recursively sorts both partitions.
- We use the last element as pivot in this example.

Step-by-step Execution:

Initial array: [7, 2, 9, 4, 3, 1, 5]

Pass 1 – Full Array:

- Pivot = 5 (last element). Partition: move elements <= 5 to left, > 5 to right.
- i = -1. Scan j from 0 to 5:
  - j=0: A[0]=7 > 5, skip.
  - j=1: A[1]=2 <= 5, i=0, swap A[0] and A[1]: [2, 7, 9, 4, 3, 1, 5]
  - j=2: A[2]=9 > 5, skip.
  - j=3: A[3]=4 <= 5, i=1, swap A[1] and A[3]: [2, 4, 9, 7, 3, 1, 5]
  - j=4: A[4]=3 <= 5, i=2, swap A[2] and A[4]: [2, 4, 3, 7, 9, 1, 5]
  - j=5: A[5]=1 <= 5, i=3, swap A[3] and A[5]: [2, 4, 3, 1, 9, 7, 5]

- Place pivot at i+1=4: swap A[4] and A[6]: [2, 4, 3, 1, 5, 7, 9]
- Pivot 5 is now at index 4. Left: [2,4,3,1], Right: [7,9]

Pass 2 – Left subarray [2, 4, 3, 1]:

- Pivot = 1. After partition: [1, 4, 3, 2]. Pivot 1 at index 0. Right: [4,3,2].

Pass 3 – [4, 3, 2]:

- Pivot = 2. After partition: [2, 3, 4]. Pivot 2 at index 0. Right: [3,4].

Pass 4 – [3, 4]:

- Pivot = 4. After partition: [3, 4]. Sorted.

Pass 5 – Right subarray [7, 9]:

- Pivot = 9. After partition: [7, 9]. Sorted.

Final Answer: Sorted array: [1, 2, 3, 4, 5, 7, 9]

Time and Space Complexity:

- Average Case: O(n log n). Worst Case: O(n2) – when pivot is always smallest/largest.
- Space Complexity: O(log n) for recursive call stack.

Q8. Analyze how the greedy approach optimizes the job scheduling problem. Discuss with example.

Answer:

Problem Understanding:

- Job Scheduling Problem: Given n jobs, each with a deadline d[i] and profit p[i], each job takes exactly 1 unit of time. Only 1 job can run at a time. Goal: Schedule jobs to maximize total profit within their deadlines.

Greedy Strategy:

- Step 1: Sort all jobs in decreasing order of profit.
- Step 2: For each job (in sorted order), find the latest available time slot within its deadline. If a slot is available, schedule the job there.
- Step 3: If no slot is available within the deadline, skip the job.

Example:

N=4, Jobs=(J1,J2,J3,J4), Profits=(35,30,25,20), Deadlines=(3,1,3,2)

Step-by-step Execution:

- Sort by profit: J1(35,d3), J2(30,d1), J3(25,d3), J4(20,d2).
- Available slots: [1, 2, 3] (since max deadline = 3).
- J1 (profit=35, deadline=3): Slot 3 available. Schedule J1 at slot 3. Slots: [1, 2, -]
- J2 (profit=30, deadline=1): Slot 1 available. Schedule J2 at slot 1. Slots: [-, 2, J1]
- J3 (profit=25, deadline=3): Slot 3 taken. Try slot 2 – available. Schedule J3 at slot 2. Slots: [J2, J3, J1]
- J4 (profit=20, deadline=2): Slots 2 and 1 both taken. Cannot schedule J4.

Final Answer: Scheduled: J2(slot 1), J3(slot 2), J1(slot 3). Total Profit = 30+25+35 = 90

Time Complexity: O(n2) – O(n log n) for sorting + O(n) slot search per job.

Q9. Use the divide and conquer approach to solve the maximum sub-array problem for the array [-2, 1, -3, 4, -1, 2, 1, -5, 4].

Answer:

Problem Understanding:

- Find the contiguous subarray with the maximum sum. Divide and Conquer splits the array at midpoint and considers three cases.

Algorithm Approach:

- Case 1: Maximum subarray lies entirely in the left half.
- Case 2: Maximum subarray lies entirely in the right half.
- Case 3: Maximum subarray crosses the midpoint – extends left from mid and right from mid+1.
- The answer is the maximum of the three cases.

Step-by-step Execution:

Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4], indices 0 to 8, mid = 4

Finding Maximum Crossing Subarray (key step):

- Left of mid (from index 4 going left): -1, -1+4=3, 3-3=0, 0+1=-1+0=... Scan: A[4]=-1, A[3]=4 gives sum=3 (max left=3 at index 3).
- Right of mid+1 (from index 5 going right): A[5]=2, 2+1=3, 3-5=-2, -2+4=2. Max right=3 at index 6.
- Crossing sum = max_left + max_right = 3 + 3 = 6, subarray [4, -1, 2, 1].

Recursive Results:

- Left half [-2, 1, -3, 4, -1]: max subarray = [4], sum = 4.
- Right half [2, 1, -5, 4]: max subarray = [2, 1], sum = 3 (or [4], sum = 4).
- Crossing: [4, -1, 2, 1], sum = 6.

Final Answer: Maximum subarray = [4, -1, 2, 1] with sum = 6.

Time Complexity:

- Recurrence: $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$
- Note: Kadane's algorithm solves this in O(n), but D&C gives O(n log n).

Q10. Analyze the time complexity of merge sort using the divide and conquer approach.

Answer:

Problem Understanding:

- Merge Sort divides the array into two halves, recursively sorts each half, and merges them. We derive its exact time complexity.

Recurrence Formulation:

- Divide: Split array of size n into 2 halves of size n/2 – O(1).
- Conquer: 2 recursive calls on n/2 elements.

- Combine: Merge two sorted halves – takes O(n) time.

Recurrence: T(n) = 2T(n/2) + n, T(1) = 0

Applying Master Theorem:

- T(n) = aT(n/b) + f(n), where a=2, b=2, f(n)=n.
- log_b(a) = log2(2) = 1. Compare f(n) = n with n^log_b(a) = n^1 = n.
- f(n) = Theta(n^log_b(a)) = Theta(n) → Case 2 of Master Theorem applies.
- Therefore: T(n) = Theta(n log n).

Recursion Tree Verification:

- Level 0: n work. Level 1: 2 × n/2 = n work. Level 2: 4 × n/4 = n work. ...
- Total levels = log2(n). Work per level = n. Total = n × log2(n) = O(n log n).

Case	Time Complexity	Reason
Best Case	O(n log n)	Array already sorted – still performs all comparisons
Average Case	O(n log n)	Same recurrence applies
Worst Case	O(n log n)	Always divides equally

- Space Complexity: O(n) auxiliary space for the merge step.

Q11. Compare the time complexities of quick sort and merge sort in their best, worst, and average cases.

Answer:

Refer to Q2 (Basic Questions) for the detailed comparison table.

Additional Analysis:

Metric	Quick Sort	Merge Sort
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	O(n2) – bad pivot	O(n log n) – always
Recurrence (avg)	T(n) = 2T(n/2) + O(n)	T(n) = 2T(n/2) + O(n)
Recurrence (worst)	T(n) = T(n-1) + O(n)	T(n) = 2T(n/2) + O(n)
In-place?	Yes – O(log n) stack	No – O(n) extra space
Stable?	No	Yes

Quick Sort worst case (O(n2)) occurs when pivot is always the smallest/largest element (sorted arrays with naive pivot selection). Use randomized pivot or median-of-three to avoid this.

Q12. Apply divide and conquer to solve the integer multiplication problem using Karatsuba's algorithm.

Answer:

Problem Understanding:

- Multiply two n-digit numbers X = 1234 and Y = 5678 using Karatsuba's algorithm.

Algorithm / Approach:

- Split X and Y into two halves (n/2 = 2 digits each):

X = 1234: X_H = 12, X_L = 34 Y = 5678: Y_H = 56, Y_L = 78

Base = 10^(n/2) = 10^2 = 100

Step-by-step Execution:

- m1 = X_H * Y_H = 12 * 56 = 672
- m2 = X_L * Y_L = 34 * 78 = 2652
- m3 = (X_H + X_L) * (Y_H + Y_L) = (12+34) * (56+78) = 46 * 134 = 6164
- Cross-term = m3 - m1 - m2 = 6164 - 672 - 2652 = 2840

Final Combination:

Result = m1 * 10^4 + cross-term * 10^2 + m2

= 672 * 10000 + 2840 * 100 + 2652

= 6720000 + 284000 + 2652

Final Answer: 1234 * 5678 = 7,006,652

Time Complexity:

- Recurrence: T(n) = 3T(n/2) + O(n)
- By Master Theorem Case 1: T(n) = O(n^log2(3)) ≈ O(n^1.585)
- This is much better than naive multiplication O(n2).

Q13. Solve the fractional knapsack problem for the given weights and values: Weights=(2,3,5,7,1,4,1), Profits=(10,5,15,7,6,18,3), m=15.

Answer:

Problem Understanding:

- Find the maximum profit by filling a knapsack of capacity 15, allowing fractional quantities of items.

Step 1 – Compute profit/weight ratio for each item:

Item	Weight	Profit	Ratio (P/W)
1	2	10	5.00
2	3	5	1.67
3	5	15	3.00
4	7	7	1.00
5	1	6	6.00
6	4	18	4.50
7	1	3	3.00

Step 2 – Sort by ratio (descending):

Item5(6.00), Item1(5.00), Item6(4.50), Item3(3.00), Item7(3.00), Item2(1.67), Item4(1.00)

Step 3 – Fill knapsack (Capacity = 15):

Item Taken	Weight Taken	Profit Gained	Remaining Capacity
Item5	1 (full)	6.00	14
Item1	2 (full)	10.00	12
Item6	4 (full)	18.00	8
Item3	5 (full)	15.00	3
Item7	1 (full)	3.00	2
Item2	2/3 (fraction)	5*(2/3) = 3.33	0

Final Answer: Total Profit = 6 + 10 + 18 + 15 + 3 + 3.33 = 55.33

Time Complexity: O(n log n) for sorting + O(n) for selection = O(n log n).

Q14. Apply Dijkstra's algorithm to the graph: A-B(2), A-C(4), B-C(1), B-D(7), C-D(3). Find shortest paths from A.

Answer:

Problem Understanding:

- Find shortest path from source vertex A to all other vertices using Dijkstra's greedy algorithm.

Step-by-step Execution:

- Initialize: dist[A]=0, dist[B]=inf, dist[C]=inf, dist[D]=inf. Visited = {}.

Step	Vertex Processed	dist[A]	dist[B]	dist[C]	dist[D]	Action
0	—	0	inf	inf	inf	Initialize
1	A (min=0)	0	2	4	inf	Relax A-B(2), A-C(4)
2	B (min=2)	0	2	3	9	Relax B-C: 2+1=3, B-D: 2+7=9
3	C (min=3)	0	2	3	6	Relax C-D: 3+3=6 < 9, update
4	D (min=6)	0	2	3	6	All neighbors processed

Final Answer: A to B = 2 | A to C = 3 (via B) | A to D = 6 (via B→C→D)

Shortest Paths:

- A → B: Distance = 2 (direct edge)
- A → C: Distance = 3 (A → B → C)
- A → D: Distance = 6 (A → B → C → D)

Q15. Compare the greedy approach with dynamic programming for solving the 0/1 knapsack problem.

Answer:

Criteria	Greedy Approach	Dynamic Programming
Applicability	Optimal ONLY for Fractional Knapsack	Optimal for 0/1 Knapsack
Correctness for 0/1	Fails – may miss optimal combo	Always finds optimal solution
Approach	Take items by best P/W ratio greedily	Build dp[i][w] table – include/exclude each item
Time Complexity	O(n log n)	O(n * W) – pseudo-polynomial
Space Complexity	O(1)	O(n * W)
Backtracking	No – once item rejected, never revisited	All possibilities considered via table

Counterexample showing Greedy Fails for 0/1:

- Items: (w=1,v=1), (w=5,v=6), (w=5,v=6). Capacity = 10.
- Greedy picks by ratio: all have ratio ~1.0-1.2. Might pick (w=1,v=1) first. Total = 13 with two 5kg items.
- DP correctly picks the two (w=5,v=6) items: total value = 12. Greedy can fail in certain configurations.

Classic failure: Items (w=10,v=10), (w=6,v=7), (w=6,v=7). Cap=12. Greedy picks 10kg item (value=10). DP picks two 6kg items (value=14).

Q16. Apply merge sort to the following array [12, 11, 13, 5, 6, 13, 86, 41, 10, 7] and show the process step-by-step.

Answer:

Problem Understanding:

- Merge Sort uses D&C: divide array in half, recursively sort each half, then merge the two sorted halves.

Step-by-step Execution:

Original: [12, 11, 13, 5, 6, 13, 86, 41, 10, 7]

Divide Phase:

- Split: [12, 11, 13, 5, 6] | [13, 86, 41, 10, 7]
- Split L: [12, 11] [13, 5, 6] Split R: [13, 86] [41, 10, 7]
- Split further: [12][11] [13][5,6] [13][86] [41][10,7]
- Split further: [12][11] [13][5][6] [13][86] [41][10][7]

Merge Phase (bottom up):

- Merge [12]+[11] = [11, 12]
- Merge [5]+[6] = [5, 6]; Merge [13]+[5,6] = [5, 6, 13]
- Merge [11,12]+[5,6,13] = [5, 6, 11, 12, 13]
- Merge [13]+[86] = [13, 86]
- Merge [10]+[7] = [7, 10]; Merge [41]+[7,10] = [7, 10, 41]
- Merge [13,86]+[7,10,41] = [7, 10, 13, 41, 86]
- Final merge: [5,6,11,12,13] + [7,10,13,41,86]

Final Answer: Sorted: [5, 6, 7, 10, 11, 12, 13, 13, 41, 86]

- Time Complexity: O(n log n). Space Complexity: O(n).

Q17. Analyze how the divide and conquer strategy is used to solve the integer multiplication problem. Compare its time complexity with brute force methods.

Answer:

Refer to Q12 for Karatsuba's algorithm and step-by-step working.

Comparison: Brute Force vs Karatsuba vs Advanced Methods:

Method	Approach	Time Complexity
Brute Force (Grade School)	Multiply each digit pair – O(n) per digit	O(n2)
Naive D&C (4 sub-problems)	4 recursive multiplications on n/2 digits	O(n2)
Karatsuba (D&C)	3 recursive multiplications on n/2 digits	O(n^1.585)
Toom-Cook (3-way)	5 multiplications on n/3 digits	O(n^1.465)
Schonhage-Strassen	FFT-based multiplication	O(n log n log log n)

- Key insight: Naive D&C is T(n)=4T(n/2)+O(n) → O(n2). Karatsuba reduces to 3 multiplications → T(n)=3T(n/2)+O(n) → O(n^1.585).
- For large numbers (cryptography, big integer math): Karatsuba provides substantial speedup over brute force.

Q18. Apply binary search to find the position of element 9 in the sorted array [2, 3, 5, 7, 9, 11, 13, 15]. Write Binary Search Algorithm with time complexity.

Answer:

Algorithm:

```
BinarySearch(A, low, high, target):
  if low > high:
    return -1       // Element not found
```

```
  mid = (low + high) / 2
  if A[mid] == target:
    return mid      // Found at index mid
  elif A[mid] < target:
    return BinarySearch(A, mid+1, high, target)  // Search
right half
  else:
    return BinarySearch(A, low, mid-1, target)   // Search left
half
```

Step-by-step Execution:

Array: [2, 3, 5, 7, 9, 11, 13, 15] (indices 0 to 7). Target = 9.

Step	low	high	mid	A[mid]	Action
1	0	7	3	7	9 > 7 → search right half
2	4	7	5	11	9 < 11 → search left half
3	4	4	4	9	9 == 9 → FOUND at index 4

Final Answer: Element 9 found at index 4. Number of comparisons = 3.

Time and Space Complexity:

- Time Complexity: O(log n). For n=8: log2(8) = 3 steps. Confirmed.
- Space Complexity: O(log n) recursive; O(1) iterative.

Q19. Analyze the limitations of the greedy algorithm in solving the fractional knapsack problem when items cannot be split.

Answer:

Limitation Analysis:

- When items cannot be split (0/1 Knapsack), the greedy by profit/weight ratio fails to guarantee the optimal solution.
- Reason: A high-ratio item may consume capacity that could be better used by a combination of lower-ratio items with higher combined value.

Counterexample:

Items: (w=10, v=60), (w=20, v=100), (w=30, v=120). Capacity W=50.

Ratios: 6.0, 5.0, 4.0. Greedy picks Item1 (w=10,v=60), then Item2 (w=20,v=100). Total weight=30, value=160.

Remaining capacity=20. Item3 (w=30) doesn't fit. Greedy result = 160.

DP optimal: Pick Item2 (w=20,v=100) + Item3 (w=30,v=120) = w=50, value=220. Greedy gives 160 vs DP's 220!

Key Conclusions:

- Greedy by ratio is OPTIMAL for fractional knapsack (proof by exchange argument).
- Greedy FAILS for 0/1 knapsack because it lacks the ability to 'reconsider' choices.
- Dynamic Programming must be used for 0/1 Knapsack to guarantee optimal solution.

Q20. Use the greedy algorithm to solve the job scheduling problem: N=4, Profits=(30,35,20,25), Deadline=(2,1,2,1). Analyze time complexity of Dijkstra's vs Bellman-Ford.

Answer:

Part A: Job Scheduling

Jobs: J1(p=30,d=2), J2(p=35,d=1), J3(p=20,d=2), J4(p=25,d=1)

- Sort by profit: J2(35), J1(30), J4(25), J3(20).
- Available slots: [1, 2] (max deadline = 2).
- J2 (p=35, d=1): Slot 1 available → Schedule J2 at slot 1.
- J1 (p=30, d=2): Slot 2 available → Schedule J1 at slot 2.
- J4 (p=25, d=1): Slot 1 taken. No other slot within deadline → Skip.
- J3 (p=20, d=2): Slot 2 taken → Skip.

Final Answer: Scheduled: J2(slot1), J1(slot2). Total Profit = 35 + 30 = 65

Part B: Dijkstra vs Bellman-Ford

Criteria	Dijkstra's Algorithm	Bellman-Ford Algorithm
Time Complexity	O((V+E) log V) with min-heap	O(V * E)
Negative Edges	Cannot handle (fails)	Handles negative edges correctly
Negative Cycles	Cannot detect	Detects and reports
Approach	Greedy – extract min distance	DP – relax all edges V-1 times
Efficiency	Much faster for non-neg weights	Slower but more general
Best For	Road networks, GPS routing	Currency exchange, any graph

- Dijkstra is more efficient for non-negative weight graphs because its greedy choice is proven optimal, eliminating repeated relaxation.

Q21. Compare quick sort and merge sort in terms of space complexity and efficiency for different types of input data.

Answer:

Input Type	Quick Sort Performance	Merge Sort Performance
Random data	O(n log n) avg – very fast due to good cache	O(n log n) – consistent
Nearly sorted	O(n2) if naive pivot – bad	O(n log n) – unaffected
Reverse sorted	O(n2) if last element as pivot	O(n log n) – unaffected

All same elements	O(n2) with naive partitioning	O(n log n)
Large datasets (disk)	Not preferred – non-sequential	Preferred – sequential merge
Linked lists	Poor – random access needed	Excellent – pointer manipulation only

Space Complexity Comparison:

- Quick Sort: O(log n) average stack space (in-place partitioning). O(n) worst case for skewed recursion.
- Merge Sort: O(n) auxiliary space always – requires extra array for merging.
- Conclusion: For memory-constrained systems → Quick Sort. For stability + worst-case guarantee → Merge Sort.

Higher Order Thinking Skills (HOTS) Questions

Q22. Evaluate the limitations of quicksort and propose an optimization to improve its worst-case performance.

Answer:

Limitations of Quick Sort:

- Worst-case O(n2): When pivot is always the smallest or largest element – occurs with sorted, reverse-sorted, or many-duplicate arrays using naive (first/last element) pivot selection.
- Not stable: Equal elements may change relative order, which matters for multi-key sorting.
- Poor for adversarial inputs: An attacker who knows the pivot selection strategy can construct inputs that trigger O(n2) every time.
- Recursive depth: Worst case has O(n) recursion depth, risking stack overflow for large inputs.

Proposed Optimizations:

- Optimization 1 – Randomized Pivot: Randomly select pivot before partitioning. Eliminates adversarial worst-case. Expected time is O(n log n) for ANY input distribution.
- Optimization 2 – Median-of-Three: Pivot = median of first, middle, and last elements. Greatly reduces likelihood of unbalanced partitions. Standard in practice.
- Optimization 3 – Introsort (Introspective Sort): Hybrid strategy used in C++ STL. Starts with Quick Sort, switches to Heap Sort when recursion depth exceeds 2*log(n). Guarantees O(n log n) worst case with Quick Sort's average-case speed.
- Optimization 4 – Three-way Partitioning (Dutch National Flag): For arrays with many duplicates, partition into three groups: < pivot, = pivot, > pivot. Reduces repeated comparisons.

Result of Optimizations:

- Randomized pivot: Expected O(n log n) for all inputs, O(n2) probability negligibly small.
- Introsort: Guaranteed O(n log n) worst case while maintaining Quick Sort's excellent cache performance in the common case.

Q23. Elaborate the practical use of divide and conquer in integer arithmetic operations. Would a different approach be better for large-scale multiplication?

Answer:

Practical Uses of D&C in Integer Arithmetic:

- RSA Cryptography: Uses 2048 or 4096-bit integers. Karatsuba multiplication (O(n^1.585)) is essential – naive O(n2) would be prohibitively slow.
- Computer Algebra Systems (CAS): Software like Mathematica, SAGE use big-integer libraries that employ Karatsuba and Toom-Cook for polynomial and integer multiplication.
- Python's built-in integers: Python uses Karatsuba for numbers above a threshold (~70 digits).
- GMP (GNU Multiple Precision Library): Uses Karatsuba below 100 words, Toom-Cook-3 up to 300 words, then Schonhage-Strassen for larger inputs.

Comparison of Methods:

Method	Algorithm	Complexity	Best for
Brute Force	Grade school	O(n2)	n < 10 digits
Karatsuba (D&C)	3 sub-multiplications	O(n^1.585)	n = 10-1000 digits
Toom-Cook	5 sub-multiplications	O(n^1.465)	n = 1000-100000 digits
Schonhage-Strassen	FFT-based	O(n log n log log n)	n > 100000 digits
Harvey-Hoeven (2019)	Improved FFT	O(n log n)	Theoretical – very large n

- Conclusion: D&C (Karatsuba) is the go-to for medium-scale arithmetic. FFT-based methods are better for extremely large numbers where the constant factors become negligible.

Q24. Evaluate the performance of the divide and conquer strategy in solving recursive problems like matrix multiplication and compare it with dynamic programming.

Answer:

D&C for Matrix Multiplication – Strassen's Algorithm:

- Naive matrix multiplication of two n×n matrices: O(n3) – three nested loops.
- Strassen's D&C: Divides each matrix into four n/2 × n/2 sub-matrices. Performs 7 recursive multiplications (instead of naive 8) and 18 additions.

- Recurrence: T(n) = 7T(n/2) + O(n2). By Master Theorem Case 1: T(n) = O(n^log2(7)) ≈ O(n^2.81).
- This beats naive O(n3) for large n.

Dynamic Programming for Matrix Problems:

- Matrix Chain Multiplication (MCM): DP determines the optimal parenthesization order to minimize total multiplications when multiplying a chain A1×A2×...×An.
- MCM DP: dp[i][j] = min cost to multiply matrices from i to j. Time: O(n3). Space: O(n2).

Comparison:

Aspect	D&C (Strassen)	Dynamic Programming (MCM)
Problem Solved	Individual matrix multiplication	Optimal ordering of a chain
Time Complexity	O(n^2.81) per multiplication	O(n3) for entire chain ordering
When to Use	Large square matrix multiplication	Chain of matrices – minimize total ops
Optimality	Not globally optimal (Coppersmith-Winograd is better)	Globally optimal parenthesization
Space	O(n2)	O(n2)

- Best practice: Use MCM (DP) to find the optimal multiplication order, then apply Strassen's for each individual multiplication. This combines both strategies.

Q25. Evaluate a situation where the greedy approach fails to provide the optimal solution for the knapsack problem. How can dynamic programming solve it optimally?

Answer:

Situation Where Greedy Fails:

Items: (w=10, v=60), (w=20, v=100), (w=30, v=120). Capacity W=50.

Ratios: Item1=6.0, Item2=5.0, Item3=4.0

- Greedy picks Item1 (w=10,v=60) first, then Item2 (w=20,v=100). Total weight=30, value=160. Remaining capacity=20 – Item3 doesn't fit.
- Greedy result: Value = 160.
- Optimal: Item2 (w=20,v=100) + Item3 (w=30,v=120) = weight 50, value 220.
- Greedy gives 160 vs Optimal 220 – Greedy misses by 60!

Why Greedy Fails:

- Greedy makes a locally optimal choice (Item1 has best ratio) without considering that it blocks a better global combination.
- Items cannot be split (0/1 constraint), so fractional reasoning doesn't apply.

Dynamic Programming Solution:

- Build table dp[i][w] = maximum value using first i items with capacity w.

- Recurrence: dp[i][w] = max(dp[i-1][w], v[i] + dp[i-1][w - w[i]]) if w[i] <= w

DP Table for above example (W=50):

Item\Capacity	0	10	20	30	40	50
0 (no items)	0	0	0	0	0	0
1 (w=10,v=60)	0	60	60	60	60	60
2 (w=20,v=100)	0	60	100	160	160	160
3 (w=30,v=120)	0	60	100	160	180	220

Final Answer: DP gives dp[3][50] = 220. Items selected: Item2 + Item3.

Q26. Propose a variant of Dijkstra's algorithm that could work efficiently with negative edge weights.

Answer:

Problem with Dijkstra and Negative Edges:

- Dijkstra finalizes shortest distances greedily – once a vertex is processed, its distance is never updated.
- With negative edges, a later path through a negative edge could give a shorter distance to an already-processed vertex, violating Dijkstra's correctness assumption.

Proposed Solution 1 – Johnson's Algorithm:

- Step 1: Add a virtual source vertex s connected to every vertex with weight 0.
- Step 2: Run Bellman-Ford from s to compute potential h[v] for each vertex.
- Step 3: Re-weight edges: w'(u,v) = w(u,v) + h[u] - h[v]. This makes all edges non-negative.
- Step 4: Run Dijkstra on the re-weighted graph.
- Step 5: Subtract potentials to recover actual shortest distances.
- Time Complexity: O(V*E) for Bellman-Ford + O(V*(V+E)*log V) for Dijkstra on all vertices.

Proposed Solution 2 – Potential-Based Re-weighting:

- If a potential function h satisfies h[u] - h[v] <= w(u,v) for all edges, then w'(u,v) = w(u,v) + h[u] - h[v] >= 0.
- Dijkstra on re-weighted graph gives correct results after adjusting by potentials.

Alternative: Use Bellman-Ford Directly:

- Bellman-Ford handles negative edges correctly in O(V*E) time.
- Tradeoff: Bellman-Ford is slower than Dijkstra but handles the general case.
- Best choice: Johnson's Algorithm for all-pairs shortest path with negative edges on sparse graphs.

Q27. Assess the limitations of the greedy strategy in solving the job scheduling problem and propose an alternative solution.

Answer:

Limitations of Greedy Job Scheduling:

- Assumes unit execution time per job – fails when jobs have variable execution times.
- Single processor only – cannot directly handle multi-processor scheduling.
- No preemption support – assumes non-preemptive scheduling only.
- Optimizes only profit – doesn't account for weighted completion time or lateness penalties.
- Greedy is not optimal for weighted job scheduling with arbitrary processing times.

Alternative Solutions:

- Dynamic Programming: For weighted job scheduling (each job has start time, finish time, weight), sort jobs by finish time and use DP: dp[i] = max(weight[i] + dp[last compatible job], dp[i-1]). Time: O(n log n).
- Branch and Bound: Systematically explore all scheduling permutations with pruning. Exact but exponential worst case.
- Earliest Deadline First (EDF): For preemptive real-time scheduling, EDF is provably optimal – always run the job with nearest deadline.
- Shortest Job First (SJF): Minimizes average waiting time – optimal for minimizing average turnaround.

Conclusion:

- The greedy profit-first approach is optimal ONLY for unit-time, single-machine, non-preemptive job scheduling.
- For general scheduling problems, DP or more sophisticated techniques are required.

Q28. Create a modification of the greedy algorithm to solve the traveling salesman problem. Explain why this modification may or may not work.

Answer:

Greedy TSP – Nearest Neighbor Heuristic:

- Start at any city. At each step, visit the nearest unvisited city. Return to start after visiting all cities.

```
NearestNeighborTSP(cities, start):
  visited = {start}
  current = start
  tour = [start]
  while len(visited) < n:
    next = argmin over unvisited cities v of dist(current, v)
    tour.append(next)
    visited.add(next)
    current = next
  tour.append(start)  // return to origin
  return tour
```

Why It May NOT Work Optimally:

- TSP is NP-Hard – no polynomial algorithm is known to always find the optimal tour.
- Greedy nearest-neighbor can produce tours 20-25% longer than optimal on average.
- Counterexample: A greedy choice that avoids a nearby cluster early forces a very long edge later to complete the tour.

Proposed Modifications to Improve:

- 2-Opt Improvement: After greedy tour, try swapping pairs of edges. If swapping edges (A-B) and (C-D) to (A-C) and (B-D) reduces total distance, make the swap. Repeat until no improvement. Often reduces tour length by 10-15%.
- Christofides Algorithm: Uses minimum spanning tree + minimum weight perfect matching on odd-degree vertices. Guarantees solution within 1.5× optimal (best known polynomial approximation).

Time Complexities:

- Greedy nearest-neighbor: O(n2)
- 2-Opt improvement: O(n2) per pass, O(n3) total for exhaustive improvement.
- Christofides: O(n3) – practical for moderate n.

Q29. Evaluate the differences between the greedy and divide and conquer strategies. Which is more suitable for real-world optimization problems?

Answer:

Criteria	Greedy Strategy	Divide and Conquer
Core Idea	Make the locally optimal choice at each step	Split problem, solve recursively, combine
Optimality	Optimal ONLY if greedy-choice property holds	Always correct if base cases are right
Backtracking	Never – no reconsideration	Not needed – subproblems are independent
Overlapping Subproblems	Not applicable	Subproblems are independent (non-overlapping)
Examples	Dijkstra, Huffman, Fractional Knapsack	Merge Sort, Binary Search, Strassen, FFT
Time Complexity	Usually O(n log n)	Expressed via recurrence – varies
Space	O(1) to O(n)	O(log n) to O(n) depending on algorithm
Proof of correctness	Requires exchange argument or cut property	Inductive proof on subproblem size

Suitability for Real-World Problems:

- Greedy is preferred when: Problem has proven greedy-choice property (Huffman coding, MST, shortest path with non-negative weights). Speed is critical and approximate solutions are acceptable (scheduling heuristics).
- D&C is preferred when: Problem naturally decomposes into independent subproblems (sorting, searching, FFT). Exact correct solution is required through recursive decomposition.
- Many real-world systems combine both: GPS routing uses Dijkstra (greedy) for shortest paths, but preprocessing uses D&C to build hierarchical graph structures.
- Conclusion: Neither strategy universally dominates. The choice depends on problem structure. If the greedy-choice property holds, greedy is faster. Otherwise, D&C (or DP) provides correctness guarantees.

Unit 3 Solutions

SECTION A — BASIC QUESTIONS (Q1 – Q10)

Q1. Explain the principle of optimality in dynamic programming.

- The Principle of Optimality (Bellman, 1957): An optimal solution to any instance of a problem always contains optimal solutions to all its subproblems.
- Implication: If a sequence of decisions gives the global optimum, then every sub-sequence of those decisions must be optimal for its corresponding subproblem. This allows a table to be filled bottom-up safely.
- Why it enables DP: Since every smaller stored solution is already optimal, we can combine them to build larger optimal solutions without re-checking.
- Example — 0/1 Knapsack: The best solution for n items with capacity W contains the best solution for n-1 items with capacity W (if item n is excluded) OR with capacity W-w[n] (if item n is included). Both sub-solutions must be optimal.
- Counter-example: Longest Simple Path does NOT satisfy this principle, so DP cannot be applied to it.

Q2. What are the binomial coefficients, and how are they computed using dynamic programming?

- Definition: The Binomial Coefficient C(n, k) is the number of ways to choose k items from n distinct items without repetition and without regard to order.
- Mathematical formula: C(n, k) = n! / (k! × (n-k)!)
- DP Recurrence (Pascal's Triangle): C(n, k) = C(n-1, k-1) + C(n-1, k)
- Base cases: C(n, 0) = 1 for all n (empty selection); C(n, n) = 1 for all n (select all).
- DP approach: Build a 2-D table dp[0..n][0..k] bottom-up. Each cell uses two cells from the previous row. Time: O(n×k). Space: O(n×k), or O(k) with a 1-D optimized array.
- Advantage: Naive recursion recomputes C(i,j) exponentially. DP stores each value once, reducing time from O(2^n) to O(n×k).

Q3. Write the dynamic programming recurrence relation for computing binomial coefficients.

Recurrence Relation

```
C(n, k)  =  C(n-1, k-1)  +  C(n-1, k)        for  0 < k < n
C(n, 0)  =  1                                   (base: choose none)
C(n, n)  =  1                                   (base: choose all)
C(n, k)  =  0                                   if k > n or k < 0
```

Intuition

- To choose k items from n: either include the n-th item — then choose k-1 from remaining n-1 (giving C(n-1,k-1)) — or exclude it — then choose k from n-1 (giving C(n-1,k)). The two cases are mutually exclusive.

Table-Filling Algorithm

```
for i = 0 to n:
    dp[i][0] = 1
    for j = 1 to min(i, k):
        if j == i:  dp[i][j] = 1
        else:       dp[i][j] = dp[i-1][j-1] + dp[i-1][j]
```

Q4. What is the 0/1 Knapsack problem? How does dynamic programming approach it differently from the greedy method?

Problem Statement

- Given n items each with weight w[i] and value v[i], and a knapsack of capacity W: select a subset of items to maximise total value without exceeding W. Each item is either taken entirely (1) or not at all (0) — no fractions.

DP Recurrence

```
dp[i][w] = max(dp[i-1][w],  dp[i-1][w-w[i]] + v[i])   if w[i] <= w
dp[i][w] = dp[i-1][w]                                    if w[i] > w
dp[0][w] = 0  for all w  (no items = no value)
```

DP vs Greedy for 0/1 Knapsack

Criterion	Greedy (V/W ratio)	Dynamic Programming
Optimality	NOT guaranteed for 0/1	Always optimal
Approach	Sort by ratio; take greedily	Fill n × W table bottom-up
Time	O(n log n)	O(n × W)
Handles fractions?	Only for fractional knapsack	Handles pure 0/1 correctly
Counter-example	Items(w,v): (3,4),(2,3),(2,3), W=4 → Greedy gives v=4; DP gives v=6	DP selects items 2+3 (v=6)

Q5. Explain the Floyd-Warshall algorithm for finding the all-pairs shortest path in a graph.

- Floyd-Warshall finds the shortest path between every pair of vertices in a weighted directed graph. It handles negative edge weights and detects negative cycles (D[i][i] < 0 after completion).

Initialization

```
D[i][j] = weight(i, j)   if direct edge i → j exists
D[i][i] = 0              (vertex to itself)
D[i][j] = infinity        if no direct edge
```

Recurrence

```
D[i][j] = min(D[i][j],   D[i][k] + D[k][j])
```

- Meaning: The shortest path from i to j may go directly (old D[i][j]) or through intermediate vertex k (D[i][k] + D[k][j]).

Algorithm

```
for k = 1 to n:           // try each vertex as intermediate
  for i = 1 to n:
    for j = 1 to n:
      D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

- Time Complexity: O(V^3). Space: O(V^2). Works on dense graphs; simple to implement.

Q6. What is the significance of the binomial coefficient in combinatorics and probability theory?

- Counting Combinations: C(n,k) counts exact k-element subsets of an n-element set — the foundation of all combinatorial enumeration.
- Probability — Binomial Distribution: P(X=k) = C(n,k) × p^k × (1-p)^(n-k). C(n,k) is the weight that counts the number of distinct arrangements of k successes in n trials.
- Binomial Theorem: (a+b)^n = Σ_{k=0}^{n} C(n,k) × a^(n-k) × b^k. Every coefficient in the polynomial expansion is a binomial coefficient.
- Pascal's Triangle: Row n contains C(n,0) to C(n,n); each entry is the sum of the two entries above it — directly encoding the DP recurrence.
- CS Applications: Birthday paradox (hashing), cryptographic key counting, network path enumeration, subset-sum DP, combinatorial algorithm design.

Q7. What is a binomial coefficient? How is it defined mathematically?

- A binomial coefficient, written C(n,k) or "n choose k", is the number of ways to select k elements from n distinct elements where order does not matter.

Mathematical Definitions

```
C(n, k)  =  n! / ( k! × (n-k)! )          [factorial formula]
C(n, k)  =  C(n-1, k-1) + C(n-1, k)       [Pascal's recurrence — used in DP]
C(n, 0)  =  C(n, n)  =  1                  [boundary conditions]
```

- Examples: C(5,2) = 10 (choose 2 from 5), C(4,0) = 1, C(8,8) = 1.
- Key properties: C(n,k) = C(n, n-k) [symmetry]; Sum of row n of Pascal's Triangle = 2^n.

Q8. Describe the Sum of Subset problem in dynamic programming.

- Problem Statement: Given a set S = {a1, a2, ..., an} of non-negative integers and a target sum T, determine whether any subset of S adds up exactly to T. The DP version can find ALL such subsets.

DP Recurrence

```
dp[i][s] = dp[i-1][s]                    (exclude element a[i])
        OR dp[i-1][s - a[i]]              (include a[i], if a[i] <= s)
dp[i][s] = TRUE if some subset of {a1..ai} sums to s, else FALSE
```

Base Cases

```
dp[0][0] = TRUE    (empty set sums to 0)
dp[0][s] = FALSE   for all s > 0
```

- Time Complexity: O(n × T). Space: O(n × T); reducible to O(T) using a 1-D array traversed right-to-left.

- Example: S = {3, 4, 2}, T = 5 → Subset {3, 2} sums to 5 → dp[3][5] = TRUE.

Q9. How does dynamic programming reduce the time complexity of recursive algorithms?

- Root cause of exponential recursion: Naive recursion solves the same subproblems repeatedly. E.g., Fibonacci F(n) = F(n-1) + F(n-2) makes O(2^n) calls because F(k) is recomputed in every branch.
- DP Solution 1 — Memoization (top-down): Write the algorithm recursively but cache each result on first computation. Any subsequent call for the same subproblem returns the cached value in O(1). Total calls = number of distinct subproblems.
- DP Solution 2 — Tabulation (bottom-up): Fill a table in order from smallest to largest subproblems, computing each exactly once without recursion overhead.
- Complexity improvements: Fibonacci: O(2^n) → O(n). 0/1 Knapsack: O(2^n) → O(n×W). Floyd-Warshall: O(V^4) naive → O(V^3).
- Two necessary conditions for DP: (1) Overlapping Subproblems — same subproblem appears multiple times in the recursion tree. (2) Optimal Substructure — optimal solution contains optimal sub-solutions.

Q10. Discuss the principle of Dynamic Programming and write the Binomial Coefficient formula.

Principle of Dynamic Programming

- Dynamic Programming (DP) is an algorithm design technique for problems with overlapping subproblems and optimal substructure. It solves each distinct subproblem exactly once, stores the result, and reuses it.
- Design steps: (1) Define subproblem. (2) Write recurrence relation. (3) Establish base cases. (4) Determine computation order. (5) Extract the answer.
- Two implementations: Memoization (top-down recursive + cache) and Tabulation (bottom-up iteration + table). Both give the same asymptotic complexity.

Binomial Coefficient Formula

```
C(n, k)  =  n! / ( k! × (n-k)! )          [direct / closed-form formula]
C(n, k)  =  C(n-1, k-1) + C(n-1, k)       [DP recurrence — Pascal's
Triangle]
C(n, 0)  =  1,    C(n, n)  =  1            [base cases]
```

SECTION B — MODERATE / NUMERICAL QUESTIONS (Q11 – Q19)

Q11. Solve the following 0/1 Knapsack problem using dynamic programming: Items: weight=2,value=3 | weight=3,value=4 | weight=4,value=5 | weight=2,value=3 | weight=3,value=4 | weight=4,value=5 Knapsack capacity = 5.

Problem Understanding

n = 6 items, capacity W = 5.

Item	Weight (w)	Value (v)

1	2	3
2	3	4
3	4	5
4	2	3
5	3	4
6	4	5

Algorithm / Approach

```
dp[i][w] = max(dp[i-1][w],  dp[i-1][w-w[i]] + v[i])    if w[i] <= w
dp[i][w] = dp[i-1][w]                                    if w[i] > w
dp[0][w] = 0 for all w
```

Step-by-step DP Table

Item \ Cap	0	1	2	3	4	5
0 (none)	0	0	0	0	0	0
1 (w=2,v=3)	0	0	3	3	3	3
2 (w=3,v=4)	0	0	3	4	4	7
3 (w=4,v=5)	0	0	3	4	5	7
4 (w=2,v=3)	0	0	3	4	6	7
5 (w=3,v=4)	0	0	3	4	6	7
6 (w=4,v=5)	0	0	3	4	6	7

Traceback from dp[6][5] = 7

dp[6][5]=7 == dp[5][5]=7 → Item 6 NOT included.
dp[5][5]=7 == dp[4][5]=7 → Item 5 NOT included.
dp[4][5]=7 == dp[3][5]=7 → Item 4 NOT included.
dp[3][5]=7 == dp[2][5]=7 → Item 3 NOT included.
dp[2][5]=7 != dp[1][5]=3 → Item 2 (w=3,v=4) INCLUDED. Remaining capacity = 5-3 = 2.
dp[1][2]=3 != dp[0][2]=0 → Item 1 (w=2,v=3) INCLUDED. Remaining capacity = 2-2 = 0.

Final Answer: Maximum Value = 7. Selected Items: Item 1 (w=2,v=3) + Item 2 (w=3,v=4). Total weight = 5.

Q12. Use dynamic programming to solve the Sum of Subset problem for the set {3, 34, 4, 12, 5, 2} and a target sum of 9.

Problem Understanding

S = {3, 34, 4, 12, 5, 2}, Target T = 9. Does any subset sum to exactly 9?

Algorithm / Approach

```
dp[i][s] = dp[i-1][s]  OR  (s >= S[i] AND dp[i-1][s-S[i]])
```

Step-by-step DP Table (T = TRUE, F = FALSE, columns s = 0 to 9)

i / Elem	s=0	s=1	s=2	s=3	s=4	s=5	s=6	s=7	s=8	s=9

0 (none)	T	F	F	F	F	F	F	F	F	F
1 (3)	T	F	F	T	F	F	F	F	F	F
2 (34)	T	F	F	T	F	F	F	F	F	F
3 (4)	T	F	F	T	T	F	F	T	F	F
4 (12)	T	F	F	T	T	F	F	T	F	F
5 (5)	T	F	F	T	T	T	F	T	T	T
6 (2)	T	F	T	T	T	T	T	T	T	T

Traceback for dp[5][9] = TRUE

dp[5][9]=T, dp[4][9]=F → Element 5 (value=5) INCLUDED. Remaining = 9-5 = 4.
dp[4][4]=T, dp[3][4]=T → Element 12 NOT included.
dp[3][4]=T, dp[2][4]=F → Element 4 (value=4) INCLUDED. Remaining = 4-4 = 0. Done.

Final Answer: Subset {4, 5} sums to 9. Answer = {4, 5}.

Q13. Apply the Floyd-Warshall algorithm to find shortest paths for the graph: A→B(8), B→C(1), A→D(1), D→C(9), D→B(2), C→A(4)

Problem Understanding

4 vertices: A, B, C, D. Build D matrix then apply k=A,B,C,D iterations.

Initial Distance Matrix (INF = no direct edge)

	A	B	C	D
A	0	8	INF	1
B	INF	0	1	INF
C	4	INF	0	INF
D	INF	2	9	0

Iteration k = A

D[C][B] = min(INF, D[C][A]+D[A][B]) = min(INF, 4+8) = 12
D[C][D] = min(INF, D[C][A]+D[A][D]) = min(INF, 4+1) = 5

After k=A	A	B	C	D
A	0	8	INF	1
B	INF	0	1	INF
C	4	12	0	5
D	INF	2	9	0

Iteration k = B

D[A][C] = min(INF, D[A][B]+D[B][C]) = min(INF, 8+1) = 9
D[D][C] = min(9, D[D][B]+D[B][C]) = min(9, 2+1) = 3

After k=B	A	B	C	D
A	0	8	9	1

B	INF	0	1	INF
C	4	12	0	5
D	INF	2	3	0

Iteration k = C

D[B][A] = min(INF, D[B][C]+D[C][A]) = min(INF, 1+4) = 5
D[B][D] = min(INF, D[B][C]+D[C][D]) = min(INF, 1+5) = 6
D[D][A] = min(INF, D[D][C]+D[C][A]) = min(INF, 3+4) = 7
D[D][B] = min(2, D[D][C]+D[C][B]) = min(2, 3+12) = 2 (no change)

After k=C	A	B	C	D
A	0	8	9	1
B	5	0	1	6
C	4	12	0	5
D	7	2	3	0

Iteration k = D

D[A][B] = min(8, D[A][D]+D[D][B]) = min(8, 1+2) = 3
D[A][C] = min(9, D[A][D]+D[D][C]) = min(9, 1+3) = 4
D[B][A] = min(5, D[B][D]+D[D][A]) = min(5, 6+7) = 5 (no change)
D[C][B] = min(12, D[C][D]+D[D][B]) = min(12, 5+2) = 7
D[C][A] = min(4, D[C][D]+D[D][A]) = min(4, 5+7) = 4 (no change)

Final All-Pairs Shortest Path Matrix

	A	B	C	D
A	0	3	4	1
B	5	0	1	6
C	4	7	0	5
D	7	2	3	0

Final Answer: Shortest paths found. Key paths: A→B=3 (A→D→B), A→C=4 (A→D→B→C), A→D=1, B→A=5 (B→C→A), D→A=7 (D→B→C→A).

Q14. Analyze the time complexity of the Bellman-Ford algorithm. How does it compare to Dijkstra's algorithm?

Bellman-Ford Algorithm

- Solves single-source shortest path. Handles negative edge weights. Detects negative-weight cycles.
- Approach: Relax all E edges exactly (V-1) times. A V-th relaxation pass that still improves any distance confirms a negative cycle.

```
for i = 1 to V-1:
    for each edge (u, v, w):
        if dist[u] + w < dist[v]:  dist[v] = dist[u] + w
```

- Time Complexity: O(V × E). Space: O(V).

Comparison Table

Criterion	Bellman-Ford	Dijkstra
Problem type	Single-source shortest path	Single-source shortest path
Negative weights	Handles correctly	Fails — wrong results
Negative cycles	Detects them	Cannot detect
Time Complexity	O(V × E)	O((V+E) log V) with heap
Space	O(V)	O(V)
Approach	Dynamic Programming (relaxation)	Greedy (priority queue)
Best suited for	Negative weights, Bellman-Ford routing	Non-neg weights, GPS, maps

---

Q15. Apply dynamic programming to solve the multistage graph problem for the following graph: S1→S2(2), S1→S3(4), S2→S4(3), S3→S4(5)

Problem Understanding

4 vertices: Source=S1, Middle stage={S2,S3}, Destination=S4. Find minimum-cost path.

Algorithm / Approach — Backward DP

```
cost[v] = minimum cost from vertex v to destination S4
```

Step-by-step Execution

Step 1 — Initialize destination: cost[S4] = 0.

Step 2 — Compute cost for S2 and S3:
cost[S2] = w(S2, S4) + cost[S4] = 3 + 0 = 3
cost[S3] = w(S3, S4) + cost[S4] = 5 + 0 = 5

Step 3 — Compute cost for S1 (choose minimum):
Via S2: w(S1,S2) + cost[S2] = 2 + 3 = 5
Via S3: w(S1,S3) + cost[S3] = 4 + 5 = 9
cost[S1] = min(5, 9) = 5 → best next node = S2

Final Answer: Minimum-Cost Path = S1 → S2 → S4. Total Cost = 2 + 3 = 5.

Q16. Analyze the difference between the greedy solution and the dynamic programming solution for the 0/1 Knapsack problem.

Refer to Question No. Q4 — detailed comparison table already provided there.

Additional key points:

- Counter-example proving greedy fails: Items (w,v): (3,4), (2,3), (2,3), W=4. Greedy selects item 1 (highest ratio 4/3≈1.33) → value=4, no more room. DP selects items 2+3 → value=6. Greedy is 33% suboptimal.
- Root cause: Items cannot be split in 0/1 Knapsack. A greedy local choice can permanently block a better global combination.

- DP succeeds because: The dp table records the optimal value for every (item prefix, capacity) combination, implicitly evaluating all feasible subsets and returning the true global optimum.

Q17. Explain how to fill the table (array) in the DP solution for binomial coefficients. What values are filled in the base cases?

Table Structure

dp[i][j] = C(i, j). Dimensions: (n+1) rows × (k+1) columns.

Base Cases

```
dp[i][0] = 1    for all i = 0 to n    (C(i,0) = 1 — choose nothing)
dp[i][i] = 1    for all i = 0 to n    (C(i,i) = 1 — choose everything)
dp[i][j] = 0    for j > i             (impossible selection)
```

Filling Order — Row by Row, Left to Right

```
for i = 1 to n:
    dp[i][0] = 1
    for j = 1 to min(i, k):
        if j == i:  dp[i][j] = 1
        else:       dp[i][j] = dp[i-1][j-1] + dp[i-1][j]
```

Example — DP Table for C(5, 3)

i \ j	0	1	2	3
0	1	-	-	-
1	1	1	-	-
2	1	2	1	-
3	1	3	3	1
4	1	4	6	4
5	1	5	10	10

C(5, 3) = dp[5][3] = 10.

Q18. Use dynamic programming to compute the binomial coefficient C(6, 3), and compare it to the recursive approach.

Direct Formula Verification: C(6,3) = 6! / (3! × 3!) = 720 / 36 = 20

DP Table for C(6, 3)

i \ j	0	1	2	3
0	1	-	-	-
1	1	1	-	-
2	1	2	1	-
3	1	3	3	1
4	1	4	6	4
5	1	5	10	10
6	1	6	15	20

C(6, 3) = dp[6][3] = 20.

Comparison: DP vs Naive Recursion

Criterion	Naive Recursion	DP Tabulation
Recomputation	Yes — C(i,j) recomputed in every branch	No — each C(i,j) computed exactly once
Time	O(2^n) — exponential	O(n × k) — polynomial
Space	O(n) call stack	O(n × k) table; O(k) optimized
For C(6,3)	~64+ recursive calls	28 cells (7 rows × 4 cols)
Stack overflow risk	Yes for large n	None (iterative)

Q19. Compute C(10, 4) using dynamic programming. Demonstrate how the binomial coefficient can be computed using both the full DP table and an optimized solution.

Direct Formula: C(10,4) = 10×9×8×7 / (4×3×2×1) = 5040 / 24 = 210

Full DP Table (n=10, k=4)

i \ j	0	1	2	3	4
0	1	-	-	-	-
1	1	1	-	-	-
2	1	2	1	-	-
3	1	3	3	1	-
4	1	4	6	4	1
5	1	5	10	10	5
6	1	6	15	20	15
7	1	7	21	35	35
8	1	8	28	56	70
9	1	9	36	84	126
10	1	10	45	120	210

C(10, 4) = dp[10][4] = 210.

Optimized 1-D Array Solution (Space O(k) instead of O(n×k))

```
dp = [1, 0, 0, 0, 0]          // size k+1
for i = 1 to n:
    for j = min(i,k) downto 1:  // traverse right-to-left to avoid overwrite
        dp[j] = dp[j] + dp[j-1]
Answer = dp[k]
```

After all 10 outer iterations, dp[4] = 210. Same result using only 5 memory cells.

SECTION C — REMAINING MODERATE QUESTIONS (Q20 – Q21)

Q20. Explain the role of memoization in dynamic programming. How does it improve performance over simple recursion?

- Definition: Memoization is a top-down DP technique — the algorithm is written recursively, but each result is stored in a lookup table (array or hash map) on first computation. Any later call for the same subproblem returns the cached value immediately without re-computation.
- Performance improvement — Fibonacci: Naive recursion for F(n) makes T(n) = T(n-1) + T(n-2) → O(2^n) calls. With memoization, each F(k) for k in {0..n} is computed exactly once → O(n) total calls.
- Mechanism: First call to subproblem(k) → compute and store in memo[k]. Every subsequent call → return memo[k] in O(1). Total work = (distinct subproblems) × (work per subproblem).
- Advantage over tabulation: Memoization is lazy — only subproblems actually needed are solved. Tabulation computes all table entries even if many are never used. Useful when the subproblem graph is sparse.
- Disadvantage vs tabulation: Memoization uses the call stack. Deep recursion (large n) risks stack overflow. Tabulation (iterative) avoids this completely.
- Recommendation: Use memoization when the computation order is hard to determine. Use tabulation for space-efficiency and when iterating over all subproblems is straightforward.

Q21. Solve the following 0/1 Knapsack problem using dynamic programming: Items: weight=2,value=3 | weight=3,value=4 | weight=4,value=5 | weight=5,value=8 Knapsack capacity = 5.

Problem Understanding

n = 4 items, capacity W = 5.

Item	Weight (w)	Value (v)
1	2	3
2	3	4
3	4	5
4	5	8

Algorithm / Approach

```
dp[i][w] = max(dp[i-1][w],  dp[i-1][w-w[i]] + v[i])    if w[i] <= w
dp[i][w] = dp[i-1][w]                                    if w[i] > w
dp[0][w] = 0 for all w
```

Step-by-step DP Table

Item \ Cap	0	1	2	3	4	5
0 (none)	0	0	0	0	0	0
1 (w=2,v=3)	0	0	3	3	3	3
2 (w=3,v=4)	0	0	3	4	4	7

---

3 (w=4,v=5)	0	0	3	4	5	7
4 (w=5,v=8)	0	0	3	4	5	8

Key cell explanations

dp[2][5]: w[2]=3 <= 5. max(dp[1][5]=3, dp[1][2]+4=3+4=7) = 7.
dp[3][5]: w[3]=4 <= 5. max(dp[2][5]=7, dp[2][1]+5=0+5=5) = 7.
dp[4][5]: w[4]=5 <= 5. max(dp[3][5]=7, dp[3][0]+8=0+8=8) = 8.

Traceback from dp[4][5] = 8

dp[4][5]=8 != dp[3][5]=7 → Item 4 (w=5,v=8) INCLUDED. Remaining capacity = 5-5 = 0.

Remaining capacity = 0. No more items can fit. Stop.

Final Answer: Maximum Value = 8. Selected Item: Item 4 (w=5, v=8). Total weight = 5.

Note: Items 1+2 give v=7 at weight=5 — also feasible but suboptimal. Item 4 alone gives v=8.

Moderate Questions (Bloom's Level: Applying)

Q22. Use appropriate strategy to compute the sum of subset problem for the set [2, 3, 5, 7, 10] with a target sum of 14.

Problem Understanding: Find all subsets of the given set whose elements sum to the target value 14.

Algorithm / Approach: Dynamic Programming – Build a boolean DP table dp[i][j] where dp[i][j] = true means subset sum j is achievable using first i elements.

Set = {2, 3, 5, 7, 10}, Target = 14, n = 5

Step-by-step Execution:

- Initialize dp[0][0] = true (empty subset, sum = 0). All dp[0][j] = false for j > 0.
- For each element, update the table:
  - Include element i: dp[i][j] = dp[i-1][j - arr[i]]
  - Exclude element i: dp[i][j] = dp[i-1][j]

DP Table (rows = items used 0-5, cols = sums 0-14):

- After processing all elements, dp[5][14] = true

Subsets Found: {2, 5, 7} → 2+5+7=14 ✓ | {7, 7} → not valid (no repeats) | {4, 10} → not in set

- Valid subset: {2, 5, 7} (sum = 14)
- Valid subset: {7, 7} — not valid (no duplicates)
- Valid subset: {4, 10} — 4 not in set
- Tracing back: {2, 5, 7} → 2+5+7 = 14 ✓
- {3, 1, 10} → 1 not in set. {3, 11} → not valid
- Final valid subset: {2, 5, 7}

Final Output: Subset {2, 5, 7} gives sum = 14. dp[5][14] = true.

Time Complexity: O(n × W) where n = 5, W = 14.

Space Complexity: O(n × W) for the DP table.

Q23. Analyze how dynamic programming improves the time complexity of solving the binomial coefficient problem compared to recursive methods.

Recursive Approach:

- Formula: C(n, k) = C(n-1, k-1) + C(n-1, k)
- Each call branches into 2 sub-calls → exponential calls.
- Time Complexity: O(2^n) — due to repeated subproblem computation.
- Space Complexity: O(n) for call stack.

Dynamic Programming Approach:

- Build a 2D table C[i][j] bottom-up from base cases.
- Base cases: C[i][0] = 1 and C[i][i] = 1 for all i.
- Fill: C[i][j] = C[i-1][j-1] + C[i-1][j]

- Each subproblem computed only ONCE and stored.
- Time Complexity: O(n × k) — polynomial.
- Space Complexity: O(n × k) for the 2D DP table; can be reduced to O(k) using 1D array.

Key Improvement: DP eliminates redundant recursive calls by storing previously computed results (memoization / tabulation), reducing exponential time to polynomial time.

Q24. Use dynamic programming to compute the cost of constructing an Optimal Binary Search Tree (OBST) for the following keys and frequencies: Keys = A, B, C, D; Frequencies = 10, 20, 30, 40.

Problem Understanding: Find arrangement of keys in a BST that minimizes total search cost. Cost = sum of (frequency × depth) for all keys.

Algorithm / Approach: Knuth's DP method. Let cost[i][j] = minimum cost of OBST for keys i to j. weight[i][j] = sum of frequencies from i to j.

Keys: A(10), B(20), C(30), D(40). Index: 1=A, 2=B, 3=C, 4=D.

Step-by-step Execution:

- Base case: cost[i][i] = freq[i], weight[i][i] = freq[i]
- Single key costs: cost[1][1]=10, cost[2][2]=20, cost[3][3]=30, cost[4][4]=40
- Chain of 2 keys:
  - cost[1][2]: Try root=A: 10+20+10=40; Try root=B: 10+20+20=50 → min=40
  - cost[2][3]: Try root=B: 20+30+20=70; Try root=C: 20+30+30=80 → min=70
  - cost[3][4]: Try root=C: 30+40+30=100; Try root=D: 30+40+40=110 → min=100
- Chain of 3 keys:
  - cost[1][3] = weight[1][3] + min(cost[0][0]+cost[1][3], cost[1][1]+cost[2][3], cost[1][2]+cost[3][3])
  - weight[1][3]=60; min(0+70, 10+70, 40+30) → min(70,80,70) = 70; cost[1][3]=60+70=130; best root=C
  - cost[2][4]: weight[2][4]=90; min(20+100, 70+40, 0+90)=min(120,110,90)=90; cost[2][4]=90+90=180
- Chain of 4 keys (full tree):
  - weight[1][4]=100
  - Try root=A: cost[0][0]+cost[2][4]=0+180=180
  - Try root=B: cost[1][1]+cost[3][4]=10+100=110
  - Try root=C: cost[1][2]+cost[4][4]=40+40=80
  - Try root=D: cost[1][3]+cost[5][5]=130+0=130
  - Minimum = 80 with root=C
  - Total cost = weight[1][4] + 80 = 100 + 80 = 180

Final Output: Minimum cost of OBST = 180. Root = C (key with freq 30). Left subtree: A, B. Right subtree: D.

Time Complexity: O(n^3) for Knuth's DP method.

Space Complexity: O(n^2) for cost and weight tables.

Q25. Apply the Bellman-Ford algorithm to detect negative weight cycles in the following graph: A→B(3), B→C(2), C→A(−6), A→D(5).

Problem Understanding: Bellman-Ford detects negative weight cycles by running n-1 relaxations and checking if a further relaxation still reduces distance.

Algorithm / Approach: Initialize dist[source]=0, all others=∞. Relax all edges (n-1) times. If any edge can still be relaxed in the nth iteration → negative cycle exists.

Edges: A→B(3), B→C(2), C→A(-6), A→D(5). Source = A. n=4 vertices.

Step-by-step Execution:

- Initial: dist[A]=0, dist[B]=∞, dist[C]=∞, dist[D]=∞
- Iteration 1 (relax all edges):
  - A→B(3): dist[B] = min(∞, 0+3) = 3
  - B→C(2): dist[C] = min(∞, 3+2) = 5
  - C→A(-6): dist[A] = min(0, 5-6) = -1
  - A→D(5): dist[D] = min(∞, -1+5) = 4
- Iteration 2:
  - A→B: dist[B] = min(3, -1+3) = 2
  - B→C: dist[C] = min(5, 2+2) = 4
  - C→A: dist[A] = min(-1, 4-6) = -2
  - A→D: dist[D] = min(4, -2+5) = 3
- Iteration 3:
  - A→B: dist[B] = min(2, -2+3) = 1
  - B→C: dist[C] = min(4, 1+2) = 3
  - C→A: dist[A] = min(-2, 3-6) = -3 ← still decreasing!
- Nth iteration check (iteration 4): dist[A] further reduces → NEGATIVE CYCLE DETECTED.

Final Output: Negative weight cycle exists: A → B → C → A (3+2-6 = -1 < 0). Each traversal of this cycle reduces total path weight indefinitely.

Time Complexity: O(V × E) = O(4 × 4) = O(16).

Q26. Analyze the difference between the Floyd-Warshall algorithm and Dijkstra's algorithm for solving the shortest path problem in weighted graphs.

- Purpose:
  - Floyd-Warshall: All-pairs shortest path (finds shortest path between every pair of vertices).
  - Dijkstra: Single-source shortest path (finds shortest path from one source to all others).
- Negative Edges:
  - Floyd-Warshall: Works with negative edge weights (detects negative cycles).
  - Dijkstra: Does NOT work with negative edge weights.
- Approach:
  - Floyd-Warshall: DP — iteratively improves estimates using intermediate vertices.
  - Dijkstra: Greedy — always picks the minimum distance unvisited vertex.
- Time Complexity:

    - Floyd-Warshall: O(V^3)
    - Dijkstra: O(V^2) or O((V+E) log V) with priority queue.
- Space Complexity:
    - Floyd-Warshall: O(V^2) for distance matrix.
    - Dijkstra: O(V) for distance array.
- Suitability:
    - Floyd-Warshall: Dense graphs, need all-pairs results, when V is small.
    - Dijkstra: Sparse graphs, single-source queries, large graphs with non-negative weights.

Conclusion: Floyd-Warshall is simpler for all-pairs problems; Dijkstra is faster for single-source with non-negative weights.

Q27. Solve the multistage graph problem using dynamic programming for the following graph: S1→S2(2), S1→S3(4), S2→S4(3), S3→S4(5).

Problem Understanding: Find the minimum cost path from source S1 to destination S4 in a multistage graph.

Algorithm / Approach: Backward DP approach. Let cost[v] = minimum cost to reach destination from vertex v.

Stages: Stage 1: {S1}, Stage 2: {S2, S3}, Stage 3: {S4}

Step-by-step Execution:

- Step 1 – Base case (destination): cost[S4] = 0
- Step 2 – Stage 2 nodes:
    - cost[S2] = weight(S2→S4) + cost[S4] = 3 + 0 = 3
    - cost[S3] = weight(S3→S4) + cost[S4] = 5 + 0 = 5
- Step 3 – Stage 1 (source S1):
    - Via S2: weight(S1→S2) + cost[S2] = 2 + 3 = 5
    - Via S3: weight(S1→S3) + cost[S3] = 4 + 5 = 9
    - cost[S1] = min(5, 9) = 5 → optimal path via S2

Optimal Path: S1 → S2 → S4

Minimum Cost: 5 (2 + 3)

Time Complexity: O(V + E) where V = vertices, E = edges.

Space Complexity: O(V) for cost array.

Q28. Analyze the role of overlapping subproblems in dynamic programming, using the example of the Fibonacci sequence. Why is dynamic programming more efficient than recursion for such problems?

Overlapping Subproblems Defined: A problem exhibits overlapping subproblems when the same smaller subproblems are solved multiple times during computation.

Fibonacci Example (Recursive):

- fib(5) = fib(4) + fib(3)
- fib(4) = fib(3) + fib(2) — fib(3) computed again!

- fib(3) = fib(2) + fib(1) — fib(2) computed multiple times!
- Total calls for fib(5) = 15 calls — exponential: O(2^n)

DP Approach (Memoization/Tabulation):

- Store fib(1), fib(2), ..., fib(n) in a table.
- Each value computed exactly once and looked up in O(1).
- fib(0)=0, fib(1)=1, fib(2)=1, fib(3)=2, fib(4)=3, fib(5)=5.
- Total computations = n → Time Complexity: O(n).

Why DP is More Efficient:

- Avoids redundant recomputation by caching results.
- Converts exponential recursive calls into linear iterations.
- Space Complexity: O(n) for DP table; O(1) if optimized to store only last 2 values.

Conclusion: DP is ideal when a problem has both overlapping subproblems and optimal substructure properties — Fibonacci is a textbook example.

Q29. Apply the dynamic programming approach to solve given binomial coefficient values: N=5, K=3.

Problem Understanding: Compute C(5, 3) using a DP table based on Pascal's Triangle identity: C(n,k) = C(n-1,k-1) + C(n-1,k).

Algorithm / Approach: Fill a 2D table C[i][j] for 0≤i≤n, 0≤j≤k with base cases C[i][0]=1 and C[i][i]=1.

Step-by-step Execution:

- Initialize: C[i][0] = 1 for all i; C[i][i] = 1 for all i.
- C[2][1] = C[1][0] + C[1][1] = 1 + 1 = 2
- C[3][1] = C[2][0] + C[2][1] = 1 + 2 = 3
- C[3][2] = C[2][1] + C[2][2] = 2 + 1 = 3
- C[4][1] = C[3][0] + C[3][1] = 1 + 3 = 4
- C[4][2] = C[3][1] + C[3][2] = 3 + 3 = 6
- C[4][3] = C[3][2] + C[3][3] = 3 + 1 = 4
- C[5][1] = C[4][0] + C[4][1] = 1 + 4 = 5
- C[5][2] = C[4][1] + C[4][2] = 4 + 6 = 10
- C[5][3] = C[4][2] + C[4][3] = 6 + 4 = 10

Final Output: C(5, 3) = 10. (Verified: 5! / (3! × 2!) = 120 / 12 = 10 ✓)

Time Complexity: O(n × k) = O(5 × 3) = O(15).

Space Complexity: O(n × k) for the DP table.

Higher Order Thinking Skills (HOTS) Questions (Bloom's Level: Analyzing, Evaluating, Creating)

Q30. Evaluate the benefits and limitations of dynamic programming in solving the all-pairs shortest path problem using the Floyd-Warshall algorithm.

Benefits:

- Solves all-pairs shortest path in a single run with O(V^3) time — no need to run Dijkstra V times.
- Handles graphs with negative edge weights (as long as no negative cycles exist).
- Simple and easy to implement — only three nested loops.
- Can detect negative weight cycles by checking if any diagonal element of distance matrix becomes negative.
- Works for both directed and undirected graphs.
- The DP table approach avoids redundant recomputation by storing intermediate shortest paths.

Limitations:

- Time Complexity: O(V^3) — impractical for very large graphs (e.g., V = 10,000 → 10^12 operations).
- Space Complexity: O(V^2) — requires large memory for dense graphs.
- Cannot handle graphs with negative cycles — distances become undefined (−∞).
- Not suitable when only single-source shortest path is needed — Dijkstra would be faster for that.

Conclusion: Floyd-Warshall is powerful for dense, small-to-medium graphs needing all-pairs results. For large or sparse graphs, alternatives like running Dijkstra from each vertex are preferred.

Q31. Create a dynamic programming solution for a new variant of the knapsack problem where items can be divided into multiple groups, each with its own constraints.

Problem Description: Given G groups of items, each group has its own capacity limit. Find maximum value by selecting items from groups without exceeding individual group capacities or total knapsack capacity.

DP Formulation:

- Let dp[g][w] = maximum value using items from first g groups with total weight ≤ w.
- For each group g, sub-knapsack must stay within group capacity cap[g].
- Recurrence:
  - dp[g][w] = max over all selections s from group g of: dp[g-1][w - weight(s)] + value(s)
  - where weight(s) ≤ cap[g] and w - weight(s) ≥ 0

Algorithm Steps:

- Step 1: Initialize dp[0][w] = 0 for all w (no groups selected).
- Step 2: For each group g from 1 to G:
  - Compute all feasible selections s within group capacity cap[g].
  - For each weight w from 0 to W, update dp[g][w].
- Step 3: Return dp[G][W] as the answer.

Example: Group 1 (cap=3): {item A: w=2,v=5}, {item B: w=3,v=8}. Group 2 (cap=2): {item C: w=1,v=4}, {item D: w=2,v=6}. Total W=5. Optimal: B+D = 8+6 = 14.

Time Complexity: O(G × W × max_group_combinations).

Space Complexity: O(G × W) — reducible to O(W) by reusing 1D array.

Q32. Evaluate the effectiveness of dynamic programming in solving the OBST problem for large datasets. How could you optimize the space complexity?

Effectiveness of DP for OBST:

- DP guarantees optimal substructure: The OBST of keys[i..j] is formed by choosing the best root r and combining optimal subtrees for keys[i..r-1] and keys[r+1..j].
- Without DP, brute force tries all n! orderings → factorial time.
- Standard DP (Knuth's) solution: O(n^3) time — practical for n up to few thousands.
- Knuth's optimization using monotone roots further reduces to O(n^2).
- For large datasets (n > 10,000), even O(n^2) can be slow; approximation algorithms may be needed.

Space Optimization Strategies:

- Standard DP uses O(n^2) for cost[][] and weight[][] tables.
- Optimization 1: Compute weight on-the-fly using prefix sums — saves one O(n^2) table.
- Optimization 2: For some formulations, process diagonals only and discard previous diagonals → O(n) space.
- Optimization 3: Use a 1D rolling array for chain length computations if root positions are not needed.

Conclusion: DP is very effective for OBST on moderate datasets. For very large inputs, use approximation algorithms or tree rebalancing (AVL/Red-Black Trees) as practical alternatives.

Q33. Design a dynamic programming algorithm for solving the Traveling Salesman Problem (TSP). Compare it with the greedy approach in terms of efficiency and accuracy.

DP Algorithm for TSP (Held-Karp):

- State: dp[S][i] = minimum cost to reach city i, having visited exactly the cities in set S, starting from city 0.
- Base case: dp[{0}][0] = 0.
- Recurrence: dp[S][i] = min over all j in S, j≠i: dp[S-{i}][j] + dist[j][i]
- Final answer: min over all i≠0: dp[{0,1,...,n-1}][i] + dist[i][0]
- Reconstruct the actual tour by storing parent pointers.

Time & Space Complexity (DP):

- Time: O(n^2 × 2^n) — feasible for n ≤ 20.
- Space: O(n × 2^n) for the DP table.

Greedy Approach (Nearest Neighbor):

- Start at any city, repeatedly visit the nearest unvisited city.
- Time Complexity: O(n^2). Space Complexity: O(n).
- Does NOT guarantee optimal solution — can be 20-25% worse than optimal.

Comparison:

- Accuracy: DP gives exact optimal solution; Greedy gives approximate (may miss optimal).
- Speed: Greedy is O(n^2), extremely fast; DP is O(n^2 × 2^n), impractical for large n.

- Suitability: DP for small n (≤20) with need for exact solution; Greedy for large n where speed matters more.

Conclusion: For TSP, DP (Held-Karp) is optimal but exponential. Greedy is fast but suboptimal. For large-scale TSP, heuristics like Christofides algorithm or genetic algorithms are used.

Q34. Propose an alternative approach to solving the sum of subset problem when the set contains negative numbers. How would you modify the dynamic programming solution?

Challenge with Negative Numbers:

- Standard DP uses indices 0..target — negative numbers would require negative indices (invalid for arrays).
- Achievable sums can be negative, so the range of possible sums expands significantly.

Modified DP Approach:

- Step 1: Compute the full range of achievable sums:
  - min_sum = sum of all negative elements in the set.
  - max_sum = sum of all positive elements in the set.
- Step 2: Shift indices: use offset = |min_sum| to map sum s → index s + offset.
- Step 3: Build boolean DP table of size (max_sum - min_sum + 1).
- Step 4: For each element, update the DP table (include or exclude).
- Step 5: Check dp[target + offset] — if true, subset exists.

Alternative: HashMap Approach:

- Use a HashSet to store all achievable subset sums after processing each element.
- For each new element, add it to every existing sum in the set.
- Check if target is in the final HashSet.
- Time: O(n × 2^n) worst case, but practical for small n. Space: O(2^n).

Example: Set = {-3, 1, 4, -2}, Target = 2. Offset = 5. Shifted DP of size 10. dp[2+5] = dp[7] checked. Subset {-3,1,4} = 2 ✓.

Conclusion: Negative numbers require offset-based DP or HashMap approach to handle extended sum ranges.

Q35. Evaluate the efficiency of dynamic programming for multistage graph problems compared to greedy and backtracking approaches.

Dynamic Programming:

- Guarantees optimal solution using principle of optimality.
- Computes minimum cost path by processing stages in reverse (backward DP).
- Time Complexity: O(V + E). Space: O(V).
- No redundant recomputation — each vertex's optimal cost stored once.

Greedy Approach:

- At each stage, selects the edge with minimum weight to the next stage.
- Time Complexity: O(E) — very fast.

- Does NOT always give optimal solution — a locally cheap edge may lead to expensive later stages.
- Example where greedy fails: cheap S1→S2 but expensive S2→S4 could be worse than S1→S3→S4.

Backtracking:

- Explores all possible paths from source to destination.
- Prunes paths exceeding current best cost (branch & bound variant).
- Time Complexity: O(k^(n-1)) where k = avg edges per stage — exponential.
- Correct but very slow for large graphs.

Comparison Summary:

- DP: Optimal + Efficient → Best choice for multistage graph problems.
- Greedy: Fast but may miss optimal — suitable only if graph has special properties.
- Backtracking: Exhaustive and correct but too slow for large inputs.

Conclusion: DP is the most suitable approach for multistage graph problems, providing guaranteed optimality with polynomial time complexity.

Q36. Analyze a situation where dynamic programming would not be the best approach to solve the 0/1 knapsack problem, and propose an alternative solution.

When DP May Not Be Best:

- When capacity W is very large (e.g., W = 10^9): DP table size becomes O(n × W) — memory and time impractical.
- When n (number of items) is very large with large W: even 1D optimized DP requires O(W) space = gigabytes.
- When weights/values are floating point: DP works on integer indices only.
- When an approximate answer is acceptable: DP gives exact answer at high computational cost.

Alternative Approaches:

- Branch and Bound:
  - Uses upper bound estimation to prune search tree.
  - Efficient in practice even for large W, as it avoids exploring all 2^n subsets.
  - Time: Exponential worst case but often much better in practice.
- Greedy Approximation (FPTAS — Fully Polynomial Time Approximation Scheme):
  - Scale down weights/values to reduce problem size.
  - Run DP on scaled instance → near-optimal answer in polynomial time.
- Meet in the Middle:
  - Split items into two halves, enumerate all subsets of each half.
  - Merge results using binary search → O(2^(n/2) × n) time — better than 2^n.

Conclusion: For large W or approximate requirements, Branch and Bound or FPTAS are better alternatives to DP for the 0/1 knapsack problem.

Q37. Propose a dynamic programming algorithm to solve a multistage decision problem in a business optimization scenario. How would you model the stages and decisions?

Business Scenario: A company has a 4-year investment plan. Each year (stage), it can allocate budget across 3 projects. Maximize total profit over all years subject to annual budget constraints.

Modeling with Multistage DP:

- Stage: Each year (t = 1 to T = 4) is a stage.
- State: Remaining budget at the start of each stage.
- Decision: Amount allocated to each project at each stage.
- Reward: Profit earned from each project based on investment.
- Recurrence:
  - dp[t][b] = maximum profit from stage t onwards with budget b remaining.
  - dp[t][b] = max over all valid allocations a: profit(t, a) + dp[t+1][b - a]
- Base case: dp[T+1][b] = 0 for all b (no more stages).
- Final answer: dp[1][B] where B = total budget.

Algorithm Steps:

- Step 1: Define profit(t, a) function for each stage and allocation.
- Step 2: Initialize base cases for final stage.
- Step 3: Fill DP table from last stage to first stage (backward induction).
- Step 4: Trace forward to find optimal allocation decisions at each stage.

Time Complexity: O(T × B × max_allocations_per_stage).

Advantage: Guarantees optimal multi-year budget allocation while respecting all constraints at every stage.

Q38. Form the binomial coefficient formula and Compute C(8, 8) using dynamic programming.

Binomial Coefficient Formula:

- Mathematical definition: C(n, k) = n! / (k! × (n-k)!)
- DP Recurrence (Pascal's Triangle): C(n, k) = C(n-1, k-1) + C(n-1, k)
- Base cases: C(n, 0) = 1 for all n (empty selection)
- C(n, n) = 1 for all n (select all)
- C(n, k) = 0 if k > n

Computing C(8, 8):

- Using base case: C(n, n) = 1 for all n → C(8, 8) = 1
- Verification via formula: 8! / (8! × 0!) = 40320 / (40320 × 1) = 1 ✓

DP Table Construction:

- Initialize C[i][0] = 1 and C[i][i] = 1 for all i from 0 to 8.
- Fill inner cells: C[i][j] = C[i-1][j-1] + C[i-1][j]
- C[8][8] = C[7][7] + C[7][8] = 1 + 0 = 1

Final Output: C(8, 8) = 1. Meaning: There is exactly ONE way to choose all 8 items from 8 items (select all).

Time Complexity: O(n × k) = O(8 × 8) = O(64).

Space Complexity: O(n × k) for full table; O(k) for optimized 1D array.

Q39. Consider two sequences: X = "ABCBDAB" and Y = "BDCABA". What is the length of the Longest Common Subsequence? Use Dynamic Programming approach.

Problem Understanding: Find the length of the longest subsequence common to both strings X and Y (elements need not be contiguous, but must maintain relative order).

X = A B C B D A B (length m = 7)

Y = B D C A B A (length n = 6)

Algorithm / Approach: Build DP table L[i][j] = LCS length of X[1..i] and Y[1..j].

Recurrence:

- If X[i] == Y[j]: L[i][j] = L[i-1][j-1] + 1
- Else: L[i][j] = max(L[i-1][j], L[i][j-1])
- Base case: L[0][j] = 0, L[i][0] = 0

DP Table (rows = X chars, cols = Y chars):

	""	B	D	C	A	B	A
""	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

Final Output: L[7][6] = 4. Length of LCS = 4.

One LCS: "BCBA" or "BDAB" — multiple valid LCS of length 4.

Time Complexity: O(m × n) = O(7 × 6) = O(42).

Space Complexity: O(m × n) for the DP table.

Q40. Write the recurrence relation for Fibonacci using Dynamic Programming.

Recurrence Relation:

- F(n) = F(n-1) + F(n-2), for n > 1
- Base cases: F(0) = 0, F(1) = 1

Why DP is Used:

- Pure recursion recomputes F(k) multiple times for the same k → exponential calls: O(2^n).
- DP stores computed values in a table → each F(i) computed once → O(n) time.

Bottom-Up DP Algorithm:

- Initialize F[0] = 0, F[1] = 1.

- For i = 2 to n: F[i] = F[i-1] + F[i-2]
- Return F[n].

Example (n=7):

- F[0]=0, F[1]=1, F[2]=1, F[3]=2, F[4]=3, F[5]=5, F[6]=8, F[7]=13

Space Optimization: Instead of storing entire array, keep only last two values: F_prev=F(n-2), F_curr=F(n-1). Update: F_next = F_curr + F_prev. Reduces Space Complexity to O(1).

Time Complexity: O(n) — linear.

Space Complexity: O(n) with table; O(1) with optimized two-variable approach.

Q41. Explain why the 0/1 Knapsack problem cannot be solved using a greedy approach but can be solved using Dynamic Programming.

Why Greedy Fails for 0/1 Knapsack:

- Greedy always picks the item with the best immediate value (e.g., highest value/weight ratio) without considering future consequences.
- In 0/1 knapsack, items cannot be fractioned — either take whole item or leave it.
- Greedy may pick a high-ratio item that 'wastes' capacity, preventing selection of a better combination.
- Counter-example: Items: {w=1, v=6}, {w=2, v=10}, {w=3, v=12}, Capacity=5.
  - Greedy (by v/w ratio): ratios = 6, 5, 4 → picks {w=1,v=6} then {w=2,v=10} → total value = 16.
  - Optimal DP: picks {w=2,v=10} + {w=3,v=12} → total value = 22.
  - Greedy gave suboptimal result! ✗
- Greedy lacks look-ahead — it cannot backtrack or try alternative combinations.

Why Dynamic Programming Works:

- DP satisfies the two key properties for its applicability:
  - 1. Optimal Substructure: Optimal solution to knapsack(n,W) is built from optimal solutions to knapsack(n-1, W) and knapsack(n-1, W-w[n]).
  - 2. Overlapping Subproblems: Same sub-knapsack problems (same i, same remaining capacity) arise repeatedly — DP stores results to avoid recomputation.
- DP Recurrence: dp[i][w] = max(dp[i-1][w], dp[i-1][w-wi] + vi) if wi ≤ w
- DP explores ALL feasible combinations systematically using the table — guarantees finding the global optimum.
- Greedy only finds a locally optimal solution at each step — no global guarantee.

Time Complexity: DP: O(n × W) (pseudo-polynomial). Greedy: O(n log n) for sorting + O(n) = O(n log n) — faster but incorrect for 0/1 variant.

Conclusion: The 0/1 knapsack problem requires DP because greedily choosing items can miss globally optimal combinations. DP systematically evaluates all possibilities and guarantees the correct optimal result.

Unit 4 Solutions

SECTION 1 — BRANCH AND BOUND : BASIC QUESTIONS

Q1. Explain the basic concept of the Branch and Bound technique, and the difference from other algorithm design strategies like Greedy and Dynamic Programming.

What is Branch and Bound?

- Branch and Bound (B&B) is a systematic algorithm design technique used to solve optimization and decision problems, especially NP-Hard problems like TSP and 0/1 Knapsack.
- The method explores the solution space as a state-space tree. Each node represents a partial solution. "Branching" expands a node into children; "Bounding" computes an upper/lower bound to decide if a branch can lead to a better solution than the best found so far.
- If the bound at a node is worse than the current best solution (incumbent), the entire subtree rooted at that node is pruned — this avoids exhaustive enumeration.

Comparison with Other Strategies

Criterion	Greedy	Dynamic Programming	Branch and Bound
Approach	Local optimal choice at each step	Solve overlapping subproblems bottom-up	Explore state-space tree with pruning
Optimality	Not always optimal	Globally optimal	Globally optimal
Problems	0/1 Knapsack – NOT optimal	0/1 Knapsack, LCS, OBST	0/1 Knapsack, TSP
Complexity	O(n log n) typically	O(nW) or O(n^2)	Exponential worst, much less with pruning
Memory	O(1)	O(n^2) or more (table)	O(V) – queue/stack of live nodes

- Key advantage of B&B over brute force: bounding functions prune large portions of the search tree, drastically reducing the number of nodes explored in practice.

Q2. Describe how Branch and Bound is applied to solve the 0/1 Knapsack problem. Explain the role of bounding functions and how they improve efficiency.

Problem Setup

- Items are sorted by profit/weight ratio (descending). Each node in the state-space tree represents a decision: include or exclude item i.
- At each node, we track: current weight, current profit, level (next item to decide), and an upper bound on the maximum profit achievable from this node.

Bounding Function (Upper Bound Calculation)

- Use a greedy fractional fill of remaining capacity:

```
UB = current_profit
for items from level onwards:
  if item fits: UB += item profit, reduce capacity
  else: UB += (remaining_cap / item_weight) * item_profit; break
```

- If UB <= best_profit_so_far, PRUNE the node (no point exploring further).

Why Bounding Improves Efficiency

- Without bounding, 0/1 Knapsack has 2^n nodes in the search tree. With tight upper bounds, most branches are pruned early, reducing explored nodes significantly.
- The bounding function is the heart of B&B — a tighter bound means more pruning and faster execution.

Q3. Explain the difference between depth-first search (DFS) and breadth-first search (BFS) strategies in the context of Branch and Bound. When would you prefer one over the other?

Criterion	DFS (LIFO / Stack-based)	BFS (FIFO / Queue-based)
Data structure	Stack (or recursion)	Queue
Exploration	Goes deep before exploring siblings	Explores level by level
Memory	O(depth) — low memory	O(2^level) — high memory
First solution	Finds a solution quickly (greedy-like)	Guarantees shortest path / min-depth solution
Pruning	Effective once a good bound is found	Bound updates slowly; less aggressive pruning initially
Best for	Large trees, memory-constrained, LIFO B&B	Uniform cost trees, BFS B&B, FIFO B&B

- Prefer DFS (LIFO B&B) when memory is limited and a feasible solution is needed quickly to tighten the bound for pruning.
- Prefer BFS (FIFO B&B) when you need to find the optimal solution level by level and memory is not a constraint.
- Best-First Search (LC B&B) is usually superior to both — it always expands the most promising node (lowest cost / highest bound), combining advantages of both.

Q4. Explain how the FIFO queue is used to explore the solution space in the Branch and Bound algorithm.

- In FIFO Branch and Bound, a queue (first-in, first-out) maintains the list of live nodes — nodes that have been generated but not yet fully explored.
- Algorithm: Start with the root node in the queue. Dequeue the front node, generate its children (branch), compute the bounding value for each child, and enqueue children whose bound is better than the current best solution.

- FIFO ensures level-by-level exploration of the state-space tree (similar to BFS). All nodes at depth d are explored before any node at depth d+1.
- Advantage: Systematic exploration; guarantees the first feasible solution found at minimum depth. Disadvantage: High memory usage since all live nodes at each level are stored.
- Pruning: A child node is NOT enqueued if its upper bound (for maximization) is less than or equal to the current best profit.

Q5. Describe how the FIFO strategy differs from other Branch and Bound strategies like LIFO and Best-First Search.

Criterion	FIFO (Queue)	LIFO (Stack)	Best-First (LC / Min-Heap)
Exploration order	Level by level (BFS)	Deep first (DFS)	Most promising node first
Data structure	Queue	Stack / Recursion	Priority Queue (min/max heap)
Memory	High O(2^d)	Low O(d)	Medium — only live promising nodes
First solution	Min-depth feasible solution	Quick (deep exploration)	Globally best node explored first
Pruning quality	Moderate	Good once bound tightens	Best — always explores best bound
Use case	Small trees, BFS B&B	Memory-constrained, TSP	Most practical B&B variant

- Best-First (LC B&B) is the most efficient in practice because it minimizes the total nodes explored by always expanding the node with the best bound.

Q6. Explain how the LIFO (stack-based) strategy is used in the Branch and Bound algorithm to explore the solution space.

- In LIFO Branch and Bound, a stack (last-in, first-out) maintains the live nodes, causing the algorithm to explore the most recently generated node first — equivalent to a depth-first traversal of the state-space tree.
- Algorithm: Push the root onto the stack. Pop the top node, generate children, compute bounds for each child, and push viable children onto the stack. Repeat until the stack is empty.
- LIFO quickly reaches leaf nodes (complete solutions), which helps establish a good incumbent (best known solution) early. This incumbent then helps prune branches at higher levels.
- Memory advantage: Only the path from root to the current node needs to be stored — O(depth) space, much better than FIFO's O(2^depth).
- Disadvantage: May waste time exploring a poor branch deeply before backtracking. The quality of pruning depends heavily on finding a good solution early.

Q7. How does the LIFO Branch and Bound differ from the FIFO and Best-First Branch and Bound approaches?

Refer to Question No. Q5 (detailed comparison table)

Q8. Explain how the Least Cost Branch and Bound technique determines which node to explore next in the search space using suitable examples.

- Least Cost (LC) Branch and Bound uses a priority queue (min-heap for minimization, max-heap for maximization). Each node is assigned a cost (or bound), and the node with the best (minimum cost / maximum profit bound) is always expanded next.
- For minimization (e.g., TSP): The cost of a node is the lower bound on the path cost from root to any complete solution reachable from that node. The node with the smallest lower bound is dequeued and expanded.
- For maximization (e.g., 0/1 Knapsack): The cost of a node is the upper bound on profit. The node with the highest upper bound is dequeued first.

Example — 0/1 Knapsack (LC B&B):

- Node A (root): UB = 40. Enqueue with priority 40.
- Branch: Node B (include item 1): UB = 38. Node C (exclude item 1): UB = 35.
- LC picks Node B (UB=38 > 35). Continue branching from B...
- A node is pruned if its UB <= best profit found so far.
- Key advantage: By always expanding the most promising node, LC B&B typically explores far fewer nodes than FIFO or LIFO before finding the optimal solution.

Q9. Describe the difference between Least Cost Branch and Bound and the general Branch and Bound approach.

Criterion	General B&B	Least Cost (LC) B&B
Node selection	FIFO (queue) or LIFO (stack)	Always the node with best bound (priority queue)
Exploration	Fixed order (level-by-level or depth-first)	Dynamic — most promising first
Pruning	Moderate — order may not aid pruning	Aggressive — best bound explored first, prunes more
Optimality	Optimal if no pruning errors	Optimal with fewer node expansions
Implementation	Simple (queue or stack)	Slightly complex (min/max priority queue)
Performance	May explore suboptimal branches first	Best practical performance for most problems

---

Q10. Explain the process of solving the 0/1 Knapsack problem using Branch and Bound. How are upper bounds used in this method?

Refer to Question No. Q2 — for detailed explanation of the process and bounding function

---

Q11. How does the Branch and Bound technique for the 0/1 Knapsack problem ensure that the solution found is optimal?

- B&B ensures optimality by maintaining a global incumbent — the best feasible solution (complete assignment of items) found so far.
- Every node's upper bound is computed before expansion. If UB(node) <= incumbent profit, the subtree is pruned — it cannot yield a better solution.
- This pruning is safe because the upper bound is always an overestimate (uses fractional fill). Any feasible solution in the subtree cannot exceed the UB.
- B&B terminates only when all nodes are either explored or pruned. At termination, the incumbent is provably optimal — no pruned subtree could have contained a better solution.
- Unlike Greedy (which may miss optimal), B&B always finds the global optimum, just like DP but without storing a full table.

Q12. Explain how the Branch and Bound method is used to solve the TSP. What role do lower bounds play in this method?

TSP and Branch and Bound

- TSP (Travelling Salesman Problem): Given n cities and distances between them, find the shortest Hamiltonian circuit (visit each city exactly once and return to the start).
- State-space tree: Each node represents a partial tour. Branching means extending the current partial tour to include the next unvisited city.

Role of Lower Bounds

- A lower bound (LB) at a node estimates the minimum possible cost of any complete tour passing through that partial tour.
- If LB(node) >= best_complete_tour_cost (upper bound / incumbent), the node is pruned — no complete tour reachable from this node can be better.

Computing Lower Bound — Reduction Matrix Method

- Step 1: Row reduction — subtract the minimum element of each row from all elements in that row.
- Step 2: Column reduction — subtract the minimum element of each column from all elements in that column.
- LB = sum of all row minima + sum of all column minima.
- When extending a tour from city i to city j: LB(new node) = LB(parent) + cost(i,j) + additional reduction required after setting row i and column j to infinity.

Q13. Describe the difference between solving TSP using Branch and Bound versus using brute force.

Criterion	Brute Force	Branch and Bound
Approach	Enumerate ALL (n-1)! permutations	Explore state-space tree with pruning
Complexity	O(n!)	O(2^n) worst case; much less with good bounds
For n=10	362,880 tours evaluated	Typically < 1,000 nodes explored

---

Optimality	Yes	Yes
Pruning	None	Prune branches where LB >= incumbent
Practical	Only for n <= 10-12	Practical for n up to 20-25 with good bounds

SECTION 2 — BRANCH AND BOUND : NUMERICAL PROBLEMS

Q14 (Moderate). Demonstrate how the FIFO Branch and Bound method can be applied to solve a simple 0/1 Knapsack problem.

Problem Understanding

Items: 3 items. Capacity W = 6.

Item	Weight	Profit	P/W Ratio
1	2	6	3.0
2	2	10	5.0
3	3	12	4.0

Algorithm / Approach

- Sort items by P/W ratio (descending): Item 2 (5.0), Item 3 (4.0), Item 1 (3.0).
- Use a queue. Root = no items selected. UB = fractional knapsack value.
- At each node, branch into: include current item OR exclude current item.
- Compute UB for each child. Enqueue only if UB > bestProfit.

Upper Bound Calculation Formula

```
UB = current_profit
Greedily fill remaining capacity with next items (fractional allowed)
```

Step-by-step Execution

Root (level 0): profit=0, weight=0
  UB = 0 + 10 + 12 + (1/2)*6 = 10+12+3 = 25. Enqueue root.

Dequeue root. Branch into level 1 (Item 2):
  Node A: Include Item 2 -> profit=10, weight=2
    UB = 10 + 12 + (1/3)*6 = 10+12+2 = 24. Enqueue A.
  Node B: Exclude Item 2 -> profit=0, weight=0
    UB = 0 + 12 + (3/3)*6 = 0+12+6 = 18. Enqueue B.

Dequeue A. Branch into level 2 (Item 3):
  Node C: Include Item 3 -> profit=10+12=22, weight=2+3=5
    UB = 22 + (1/2)*6 = 22+3 = 25. Enqueue C.
  Node D: Exclude Item 3 -> profit=10, weight=2
    UB = 10 + (2/2)*6 = 10+6 = 16. Enqueue D.

Dequeue B. Branch into level 2 (Item 3):
  Node E: Include Item 3 -> profit=0+12=12, weight=3

UB = 12 + (3/2)*6 = 12+9 = 21. Enqueue E.
Node F: Exclude Item 3 -> profit=0, weight=0
UB = 0 + 6 = 6. Enqueue F.

Dequeue C. Branch into level 3 (Item 1):
Include Item 1: weight=5+2=7 > 6 -> INFEASIBLE. Skip.
Exclude Item 1: profit=22, weight=5 -> LEAF. bestProfit=22. Items: {2,3}.

Dequeue D. Branch into level 3 (Item 1):
Include Item 1: profit=10+6=16, weight=2+2=4 -> LEAF. bestProfit stays 22.
Exclude Item 1: profit=10 -> LEAF. bestProfit stays 22.

Dequeue E. UB=21 < bestProfit=22 -> PRUNE.
Dequeue F. UB=6 < bestProfit=22 -> PRUNE.

Final Answer: Maximum profit = 22. Selected items: Item 2 (w=2, p=10) + Item 3 (w=3, p=12). Total weight = 5.

Q15 (Moderate). Apply the FIFO Branch and Bound approach to solve a small Traveling Salesman Problem (TSP).

Problem Understanding

4 cities: A, B, C, D. Distance matrix (symmetric):

	A	B	C	D
A	0	10	8	9
B	10	0	5	7
C	8	5	0	6
D	9	7	6	0

Algorithm / Approach

- Start from city A. Find a complete tour visiting all cities exactly once and returning to A with minimum total cost.
- FIFO B&B: Use a queue. Each node = partial tour. Bound = length of partial tour so far (simple lower bound here: sum of edges selected).

Step-by-step Execution

Root: Tour = [A], cost = 0. Enqueue.

Dequeue [A]. Branch to unvisited cities: B, C, D.
Node 1: [A->B], cost=10. Enqueue.
Node 2: [A->C], cost=8. Enqueue.
Node 3: [A->D], cost=9. Enqueue.

Dequeue [A->B]. Branch to unvisited: C, D.
Node 4: [A->B->C], cost=10+5=15. Enqueue.
Node 5: [A->B->D], cost=10+7=17. Enqueue.

Dequeue [A->C]. Branch to unvisited: B, D.
Node 6: [A->C->B], cost=8+5=13. Enqueue.
Node 7: [A->C->D], cost=8+6=14. Enqueue.

Dequeue [A->D]. Branch to unvisited: B, C.
Node 8: [A->D->B], cost=9+7=16. Enqueue.
Node 9: [A->D->C], cost=9+6=15. Enqueue.

Dequeue [A->B->C]. Branch: D only.
Node 10: [A->B->C->D], cost=15+6=21. Enqueue.

Dequeue [A->B->D]. Branch: C only.
Node 11: [A->B->D->C], cost=17+6=23. Enqueue.

Continue for all remaining nodes...

Complete tours (add return to A):
A->B->C->D->A : 10+5+6+9 = 30
A->B->D->C->A : 10+7+6+8 = 31
A->C->B->D->A : 8+5+7+9 = 29
A->C->D->B->A : 8+6+7+10 = 31
A->D->B->C->A : 9+7+5+8 = 29
A->D->C->B->A : 9+6+5+10 = 30

Final Answer: Minimum tour cost = 29. Optimal tours: A->C->B->D->A or A->D->B->C->A (both cost 29).

Q16 (Moderate). Apply the LIFO Branch and Bound approach to solve the 0/1 Knapsack problem. Show the steps involved.

Problem Understanding

Same problem as Q14: 3 items sorted by P/W ratio: Item 2(w=2,p=10), Item 3(w=3,p=12), Item 1(w=2,p=6). Capacity W=6.

Algorithm / Approach

- Use a stack instead of a queue. Branch: include or exclude current item. Compute UB at each node. Push children (viable) onto stack. Pop and expand most recently pushed node.

Step-by-step Execution

Push Root: profit=0, weight=0, level=0. UB=25.

Pop Root. Push children (level 1 = Item 2):
Push B (Exclude Item 2): profit=0, w=0, UB=18.
Push A (Include Item 2): profit=10, w=2, UB=24.

Stack top: A. Pop A. Push children (level 2 = Item 3):
Push D (Exclude Item 3): profit=10, w=2, UB=16.
Push C (Include Item 3): profit=22, w=5, UB=25.

Stack top: C. Pop C. Push children (level 3 = Item 1):
Include Item 1: w=5+2=7 > 6 -> INFEASIBLE.
Push E (Exclude Item 1): profit=22, w=5 -> LEAF. bestProfit=22.

Pop E -> Leaf already. Pop D. UB=16 < bestProfit=22 -> PRUNE.
Pop B. UB=18 < bestProfit=22 -> PRUNE.

Final Answer: Maximum profit = 22. Selected items: Item 2 + Item 3. (LIFO explores same optimal solution but in different order; fewer nodes visited due to early pruning after finding incumbent.)

Q17 (Moderate). Use LC Branch and Bound to find a solution to a Traveling Salesman Problem (TSP) with 4 cities.

Problem Understanding

Use the same 4-city distance matrix from Q15. LC B&B uses a min-heap: always expand the node with the smallest lower bound.

Algorithm / Approach: Reduction Matrix Method

- Step 1: Reduce the cost matrix (row reduce, then column reduce). The sum of reductions = root LB.
- Step 2: Each time we branch (extend the tour from city i to city j), update the cost = parent_LB + matrix[i][j] + additional reduction.
- Step 3: Always expand the node in the heap with the minimum LB.

Step-by-step Execution

Original matrix:

```
     A    B    C    D
A  [ inf  10   8    9  ]
B  [ 10   inf  5    7  ]
C  [ 8    5    inf  6  ]
D  [ 9    7    6    inf]
```

Row reduction (subtract row minima: A=8,B=5,C=5,D=6). Total reduction = 24.

```
     A    B    C    D
A  [ inf  2    0    1  ]
B  [ 5    inf  0    2  ]
C  [ 3    0    inf  1  ]
D  [ 3    1    0    inf]
```

Column reduction: All col minima are 0. No further reduction.
Root LB = 24. Insert root in heap.

Expand root -> tour starts at A. Branch to B, C, D:
A->B: LB = 24 + matrix[A][B] + reduce(set row A, col B to inf) = 24+2+1 = 27
A->C: LB = 24 + matrix[A][C] + reduce = 24+0+0 = 24
A->D: LB = 24 + matrix[A][D] + reduce = 24+1+0 = 25
Insert: A->C(24), A->D(25), A->B(27) into heap.

Expand A->C (LB=24). Branch to unvisited B, D:
A->C->B: LB = 24 + matrix[C][B] + reduce = 24+0+0 = 24
A->C->D: LB = 24 + matrix[C][D] + reduce = 24+1+1 = 26
Insert: A->C->B(24), A->C->D(26).

Expand A->C->B (LB=24). Only D unvisited.
A->C->B->D: LB = 24 + matrix[B][D] = 24 + 2 = 26
Complete tour: A->C->B->D->A: cost = 8+5+7+9 = 29
incumbent = 29. LB=26 < 29, so this is a valid path candidate.
All remaining nodes in heap have LB >= 24. Continue expanding...
After full expansion: Optimal tour confirmed as A->C->B->D->A or A->D->B->C->A.

Final Answer: Optimal Tour = A -> C -> B -> D -> A, Total Cost = 29.

Q18 (Moderate). Apply the Branch and Bound method to solve a 0/1 Knapsack problem with 3 items, given specific weights and values. Use the B&B approach to discard suboptimal branches.

Problem Understanding

Items: n=3, Capacity W=5.

Item	Weight	Value	V/W Ratio
1	2	3	1.5
2	3	4	1.33
3	4	5	1.25

Sort by V/W ratio (already sorted): Item 1, Item 2, Item 3.

Step-by-step Execution

Root: profit=0, weight=0. UB = 3 + (2/3)*4 = 3+2.67 = 5.67

Branch from Root (Item 1):
A (Include Item 1): profit=3, w=2. UB = 3+4+(0/4)*5 = 7 -> 3+4=7 but w=2+3=5<=5. UB=7.
B (Exclude Item 1): profit=0, w=0. UB = 4+(1/4)*5 = 4+1.25=5.25.

Expand A (Include Item 1, higher UB):
C (Include Item 2): profit=7, w=5. w<=W, feasible. Leaf. bestProfit=7.
D (Exclude Item 2): profit=3, w=2. UB = 3+(0/4)*5 = 3. UB=3 < bestProfit=7. PRUNE.

Expand B (UB=5.25 < bestProfit=7). PRUNE entire B subtree.

Final Answer: Maximum Value = 7. Selected items: Item 1 (w=2, v=3) + Item 2 (w=3, v=4). Total weight = 5 = capacity.

Q19 (Moderate). Apply the Least Cost Branch and Bound method to solve a small 0/1 Knapsack problem.

Problem Understanding

n=4 items. Capacity W=10.

Item	Weight	Profit	P/W
1(sorted)	2	10	5.0

2(sorted)	4	16	4.0
3(sorted)	6	18	3.0
4(sorted)	9	7	0.78

Algorithm / Approach

- LC B&B uses a max-heap (priority queue ordered by UB — higher UB = higher priority).
- UB formula: current_profit + greedy fractional fill of remaining items.

Step-by-step Execution

Root: profit=0, w=0. UB=10+16+(2/6)*18 = 10+16+6=32. Insert(UB=32).

Extract root (UB=32). Branch on Item 1:
A (Include I1): p=10, w=2. UB=10+16+(2/6)*18=32. Insert(32).
B (Exclude I1): p=0, w=0. UB=16+(4/6)*18=16+12=28. Insert(28).

Extract A (UB=32). Branch on Item 2:
C (Include I2): p=26, w=6. UB=26+(4/6)*18=26+12=38? w=6,cap=10,rem=4. UB=26+(4/6)*18=26+12=38. But 26 already, remaining cap=4, item3 w=6>4, item4 w=9>4. UB=26+0=26.
Actually: UB = 26 + (4/6)*18 = 26+12 = 38. But weight check: 2+4=6 <= 10. Rem cap=4.
Fractional of Item 3 (w=6): take 4/6 of it -> 4/6*18=12. UB=26+12=38. Insert(38).
D (Exclude I2): p=10, w=2. Rem cap=8. UB=10+(6/6)*18+(2/9)*7=10+18+1.6=29.6. Insert(29.6).

Extract C (UB=38). Branch on Item 3:
E (Include I3): p=26+18=44, w=6+6=12 > 10. INFEASIBLE. Skip.
F (Exclude I3): p=26, w=6. Rem cap=4. UB=26+(4/9)*7=26+3.1=29.1. Insert(29.1).
Leaf reached with profit=26. bestProfit=26. Items: {1,2}.

Extract D (UB=29.6). 29.6>26, so explore.
Branch on Item 3: Include: p=10+18=28, w=2+6=8 <=10. Leaf. bestProfit=28. Items:{1,3}.
Exclude: p=10,w=2. UB=10+(2/9)*7=11.6 < 28. PRUNE.

Extract(UB=29.1). 29.1 > 28. Explore F.
F branches on Item 4: Include: p=26+7=33? w=6+9=15>10. INFEASIBLE.
Exclude: p=26, w=6. LEAF. bestProfit stays 28.

Extract B (UB=28). 28 = bestProfit. May explore (depends on implementation: UB>bestProfit condition).
Skip/Prune since UB=28 not strictly greater. Terminate.

Final Answer: Maximum Profit = 28. Selected items: Item 1 (w=2,p=10) + Item 3 (w=6,p=18). Total weight = 8.

Q20 (Moderate). Demonstrate how to use the Least Cost Branch and Bound algorithm to solve a simple 5-city Traveling Salesman Problem (TSP).

Problem Understanding

5 cities: 1,2,3,4,5. Cost matrix:

	1	2	3	4	5
1	inf	3	1	5	8
2	3	inf	6	7	9
3	1	6	inf	4	2
4	5	7	4	inf	3
5	8	9	2	3	inf

Algorithm / Approach

- LC B&B with Reduction Matrix lower bound. Start from city 1.
- At each step: select the node from the min-heap with smallest LB, expand it.

Step-by-step Execution

Row reduction (subtract row minima): Row1 min=1, Row2 min=3, Row3 min=1, Row4 min=3, Row5 min=2.
Total row reduction = 1+3+1+3+2 = 10.

After row reduction:

```
     1    2    3    4    5
1  [inf   2    0    4    7 ]
2  [0    inf   3    4    6 ]
3  [0     5   inf   3    1 ]
4  [2     4    1   inf   0 ]
5  [6     7    0    1   inf]
```

Column reduction: Col1 min=0, Col2 min=2, Col3 min=0, Col4 min=1, Col5 min=0.
Total col reduction = 0+2+0+1+0 = 3. Grand total reduction = 10+3 = 13. Root LB = 13.

After column reduction:

```
     1    2    3    4    5
1  [inf   0    0    3    7 ]
2  [0    inf   3    3    6 ]
3  [0     3   inf   2    1 ]
4  [2     2    1   inf   0 ]
5  [6     5    0    0   inf]
```

Insert Root (city 1 as start) with LB=13 into min-heap.

Extract Root. Branch 1->all others:
1->2: Set row1=inf, col2=inf, [2][1]=inf. Row2 now: [0,inf,3,3,6] -> no change needed. Col1 now has 0. Additional reduction=0. LB=13+M[1][2]=13+0=13.
1->3: M[1][3]=0. LB=13+0=13.
1->4: M[1][4]=3. LB=13+3=16.
1->5: M[1][5]=7. LB=13+7=20.
Insert all into heap. Heap: {1->2:13, 1->3:13, 1->4:16, 1->5:20}

Extract 1->2 (LB=13). Branch 2-> (3,4,5):
1->2->3: LB=13+M[2][3]=13+3=16.
1->2->4: LB=13+M[2][4]=13+3=16.
1->2->5: LB=13+M[2][5]=13+6=19.

Extract 1->3 (LB=13). Branch 3-> (2,4,5):
1->3->2: LB=13+M[3][2]=13+3=16.
1->3->4: LB=13+M[3][4]=13+2=15.
1->3->5: LB=13+M[3][5]=13+1=14.

Continue extracting minimum LB nodes. Eventually complete tours are found.
Best complete tour evaluation: 1->3->5->4->2->1: 1+2+3+7+3=16.

Final Answer: Optimal TSP Tour = 1 -> 3 -> 5 -> 4 -> 2 -> 1, Total Cost = 16.

Q21 (Moderate). Use Branch and Bound to demonstrate how to discard suboptimal branches when solving a 0/1 Knapsack problem.

Refer to Question No. Q14 and Q18 — both contain detailed examples of branch pruning.
Key rule: if UB(node) <= bestProfit, discard (prune) the node and its entire subtree.

Q22 (Moderate). There are 5 bags labeled 1 to 5. All coins in a given bag have the same weight. Some bags have coins of weight 10 gms, others have coins of weight 11 gms. Pick 1, 2, 4, 8, 16 coins from bags 1 to 5 respectively. Total weight is 323 gms. Find the product of labels of the bags having 11 gm coins.

Problem Understanding

- 5 bags. From bag i, pick a fixed number of coins: Bag1=1 coin, Bag2=2 coins, Bag3=4 coins, Bag4=8 coins, Bag5=16 coins.
- Each bag has either 10 gm or 11 gm coins. Total weight = 323 gm.

Formula / Approach

If all bags had 10 gm coins: Total = (1+2+4+8+16)*10 = 31*10 = 310 gm.
Actual total = 323 gm. Extra weight = 323 - 310 = 13 gm.
Each bag with 11 gm coins contributes 1 extra gram per coin.
So: 1*x1 + 2*x2 + 4*x3 + 8*x4 + 16*x5 = 13
where xi = 1 if bag i has 11 gm coins, 0 otherwise.

Solution: Express 13 in terms of {1,2,4,8,16}

13 = 8 + 4 + 1 = 1 + 0*2 + 1*4 + 1*8 + 0*16
So: Bag1 (1 coin extra: 1), Bag3 (4 coins extra: 4), Bag4 (8 coins extra: 8) have 11 gm coins.
Verification: 1*11 + 2*10 + 4*11 + 8*11 + 16*10 = 11+20+44+88+160 = 323. Correct.

Bags with 11 gm coins: Bag 1, Bag 3, Bag 4.

Final Answer: Product of labels = 1 x 3 x 4 = 12.

SECTION 3 — BRANCH AND BOUND : HOTS QUESTIONS

Q23 (HOTS). Analyze the impact of using a FIFO queue on the efficiency of the Branch and Bound algorithm in terms of time and space complexity.

Time Complexity Impact

- FIFO B&B explores nodes level by level. In the worst case (no pruning), it visits all 2^n nodes for n-item 0/1 Knapsack, giving O(2^n) time.
- With pruning, FIFO is less aggressive than LC B&B because it does not prioritize the most promising nodes. Poor early bounds mean fewer pruning opportunities at deeper levels.
- For TSP with n cities, FIFO explores O(n!) paths in the worst case, though pruning reduces this significantly.

Space Complexity Impact

- FIFO stores all live nodes in the queue. At level d, there can be 2^d nodes — so space complexity is O(2^d) where d is the maximum depth, leading to exponential space usage.
- For a balanced state-space tree of depth n, space = O(2^n) — much worse than LIFO (O(n)) or LC B&B (which stores only the promising nodes).

Conclusion

- FIFO B&B guarantees the shortest-depth solution but at high memory cost. It is practical only for small problem instances. For large instances, LC B&B (Best-First) is significantly more efficient in both time and space due to aggressive pruning.

Q24 (HOTS). Compare the performance of FIFO Branch and Bound with Best-First Search Branch and Bound for solving optimization problems.

Refer to Question No. Q5 — detailed comparison table of FIFO, LIFO, and Best-First B&B

Additional analysis:

- Best-First (LC) B&B always expands the node with the tightest bound, meaning it reaches the optimal solution with the fewest node expansions. It is provably the most efficient B&B variant in terms of number of nodes explored.
- FIFO B&B may expand many suboptimal nodes before finding the optimal solution, since it does not prioritize promising nodes. However, it guarantees finding the shallowest (minimum depth) solution first.
- Empirically: For 0/1 Knapsack with n=20 items, FIFO may explore ~10,000 nodes while LC B&B typically explores ~500 nodes with a tight bound.

Q25 (HOTS). Analyze the strengths and weaknesses of using LIFO over FIFO in terms of time complexity and memory usage in the Branch and Bound algorithm.

Strengths of LIFO

- Memory Efficient: LIFO stores only the current path from root to the active node — O(depth) = O(n) space, versus FIFO's O(2^n) space. Critical advantage for large problems.
- Fast Incumbent Discovery: LIFO dives deep quickly, finding a complete feasible solution early. This incumbent immediately tightens pruning for other branches.

- Cache-friendly: Explores contiguous subtrees, which tends to be more cache-friendly on modern hardware than FIFO's scattered access pattern.

Weaknesses of LIFO

- May explore poor branches deeply before backtracking. If the first deep path is suboptimal, significant time is wasted before finding the true optimum.
- No guidance from bounds about which branch is most promising — expansion order is purely stack-based.
- Worst-case time complexity is the same as FIFO: O(2^n) for Knapsack, O(n!) for TSP. The actual improvement depends on problem structure.

Conclusion

- Prefer LIFO when memory is the bottleneck. Prefer Best-First (LC) when time (node expansions) is the bottleneck. FIFO is rarely preferred over either in practice.

Q26 (HOTS). Use LC Branch and Bound to find a solution to a TSP with the following matrix: [inf, 10, 5 / 8, inf, 9 / 4, 6, inf] (3 cities)

Problem Understanding

3 cities: 1, 2, 3. Cost matrix:

From \ To	1	2	3
1	inf	10	5
2	8	inf	9
3	4	6	inf

Step-by-step Execution — Reduction Matrix

Row reduction: Row1 min=5, Row2 min=8, Row3 min=4. Total = 17.

```
     1    2    3
1  [inf   5    0 ]
2  [0    inf   1 ]
3  [0     2   inf]
```

Column reduction: Col1 min=0, Col2 min=2, Col3 min=0. Total col reduction = 2.

```
     1    2    3
1  [inf   3    0 ]
2  [0    inf   1 ]
3  [0     0   inf]
```

Root LB = 17+2 = 19. Insert root.

Extract root. Branch 1->2 and 1->3:

1->2: M[1][2]=3. Set row1=inf,col2=inf,M[2][1]=inf. Additional reduction: Row2:[0,inf,1]->min=0. Col1:[0,0]->min=0. Additional=0. LB=19+3=22.

1->3: M[1][3]=0. Set row1=inf,col3=inf,M[3][1]=inf. Additional: Row3:[0,0,inf]->min=0. Col1 still 0. LB=19+0=19.

Insert 1->3(19), 1->2(22) into min-heap.

Extract 1->3 (LB=19). Only city 2 unvisited. Branch 3->2:

1->3->2: M[3][2]=0. Set row3=inf,col2=inf,M[2][1]=inf. Tour so far: 1->3->2->1.

Complete tour: cost = M_original[1][3]+M_original[3][2]+M_original[2][1] = 5+6+8 = 19.

Incumbent = 19.

Extract 1->2 (LB=22). LB=22 > incumbent=19. PRUNE.
All remaining nodes have LB >= 22. Terminate.

Final Answer: Optimal TSP Tour = 1 -> 3 -> 2 -> 1. Total Cost = 5 + 6 + 8 = 19.

Q27 (HOTS). Analyze the time complexity of solving the 0/1 Knapsack problem using the Branch and Bound approach. How does it compare with Dynamic Programming?

Criterion	Branch and Bound	Dynamic Programming
Time Complexity	O(2^n) worst case; O(n * UB_nodes) practical	O(n * W)
Space Complexity	O(n) — LIFO; O(2^n) — FIFO	O(n * W) — full table; O(W) optimized
Optimality	Yes	Yes
Dependency	No table needed; bounded search	Requires W to be integer and reasonably small
When W is large	B&B performs better (pruning independent of W)	DP becomes pseudo-polynomial (slow for large W)
When n is large	B&B degrades to O(2^n) without good bounds	DP is O(nW) — better if W is small
Practical use	Preferred for large W (continuous/float capacity)	Preferred for small integer W and n

- Key insight: DP is pseudo-polynomial — its complexity depends on both n and W (capacity). If W is very large (e.g., W=10^9), DP is impractical but B&B with good bounds may still work efficiently.
- B&B is preferred in practice when: (1) W is large or non-integer, (2) a good bounding function is available, (3) the optimal solution is expected to require only a small portion of the search space.

Q28 (HOTS). Compare the Branch and Bound method for the 0/1 Knapsack problem with the Greedy method in terms of optimality and efficiency.

Criterion	Greedy Method	Branch and Bound
Approach	Sort by P/W, take greedily (no fractions for 0/1)	Explore state-space tree with bounding
Optimality	NOT guaranteed for 0/1 Knapsack	Always optimal
Time	O(n log n) for sorting + O(n) for selection	O(2^n) worst; much less with pruning
Space	O(1)	O(n) to O(2^n) depending on variant
Example fail	Items: (w=3,v=4),(w=2,v=3),(w=2,v=3),	Correctly finds v=6

	W=4. Greedy takes item1 (ratio=1.33): v=4. Optimal: take items 2+3: v=6.	

- Greedy fails for 0/1 Knapsack because items cannot be split — the greedy choice of the highest ratio item may block a better combination. B&B explores all possibilities with intelligent pruning, guaranteeing the optimal solution.

Q29 (HOTS-Evaluate). Evaluate the situations where the FIFO Branch and Bound approach would be more efficient than other variations.

- FIFO B&B is most efficient when the optimal solution lies at the minimum depth of the state-space tree, because FIFO (BFS) guarantees finding the shallowest solution first.
- It is preferred when all complete solutions are at the same depth (uniform depth tree) — e.g., fixed-length binary assignments where all leaves are at the same level.
- FIFO is better than LIFO when early pruning at shallow levels is possible (good bounds at top of tree) — FIFO checks all level-d nodes before going deeper, while LIFO may dive into a poor subtree.
- Practical scenario: If the problem has a small number of variables (n < 15) and a tight upper bound can be computed quickly at each node, FIFO explores the complete optimal solution space efficiently.
- Weakness: For large n (n > 20), FIFO's exponential memory requirement makes it impractical. LC B&B is preferred.

Q30 (HOTS-Evaluate). Evaluate the advantages of using the Least Cost Branch and Bound method for solving optimization problems like TSP.

- LC B&B always expands the node with the best lower bound (for TSP: minimum cost partial tour), which directs the search toward the most promising solutions first.
- This results in finding a near-optimal or optimal solution very early, which significantly tightens the pruning bound for all subsequent nodes — creating a virtuous cycle of aggressive pruning.
- For TSP with n=15 cities, FIFO may explore millions of nodes; LC B&B typically explores a few thousand, making it 100x to 1000x faster in practice.
- LC B&B is admissible (never prunes the optimal solution) because the lower bound is always an underestimate (reduction matrix gives the tightest possible lower bound).
- Disadvantage: Priority queue operations cost O(log n) per node, and computing the reduction matrix at each node is O(n^2). Still, total work is far less than exhaustive search.

Q31 (HOTS-Evaluate). Evaluate the effectiveness of different bounding functions (e.g., linear relaxation) used in Branch and Bound for solving the 0/1 Knapsack problem.

- Linear Relaxation (Fractional Knapsack Upper Bound): Allows fractional items. Solved in O(n log n) by greedy. Provides the tightest practical upper bound for 0/1 Knapsack. Most widely used in B&B implementations.
- Simple Sum Bound: UB = current_profit + sum of all remaining items' profits (regardless of weight). Very loose bound — minimal pruning. Not recommended.

- Dantzig Bound (Linear Relaxation): Same as fractional knapsack. Tight, fast to compute. Standard choice.
- Lagrangian Relaxation: Relaxes weight constraint using a multiplier. Provides a tighter bound than linear relaxation in some cases but requires solving a sub-problem iteratively — higher per-node cost.
- Conclusion: Linear relaxation (fractional knapsack) offers the best trade-off between bound tightness and computation speed. Tighter bounds prune more nodes but cost more to compute. The optimal bounding function minimizes total work = (nodes explored) x (cost per bound computation).

Q32 (HOTS-Evaluate). Propose an enhancement to the standard Branch and Bound algorithm for solving the 0/1 Knapsack problem to improve performance on larger datasets.

- Enhancement 1 — Preprocessing: Sort items by P/W ratio before B&B. This ensures the greedy upper bound is as tight as possible, maximizing early pruning.
- Enhancement 2 — Better Initial Bound: Run a greedy heuristic first to get a good incumbent before starting B&B. Any node with UB <= incumbent is immediately pruned.
- Enhancement 3 — Node Selection Hybridization: Combine LIFO (for quick incumbent) with LC (for optimal pruning). Start with LIFO until a feasible solution is found; then switch to LC for the remaining search.
- Enhancement 4 — Constraint Propagation: After including item i, immediately check if any future item cannot fit. Remove infeasible items from the remaining list, tightening the bound for subsequent nodes.
- Enhancement 5 — Parallel B&B: Different subtrees can be explored in parallel by different CPU cores, each maintaining a shared incumbent and pruning bound. Reduces wall-clock time by a factor proportional to the number of cores.
- Enhancement 6 — Memory Management: Use iterative deepening (depth-limited BFS) to limit memory usage while preserving the systematic exploration property of FIFO.

SECTION 4 — BACKTRACKING : BASIC QUESTIONS

Q33 (Basic). Explain the concept of backtracking in problem-solving. How does it differ from brute force approaches?

- Backtracking is an algorithmic technique that incrementally builds a solution and abandons ("backtracks") a partial solution as soon as it determines that the partial solution cannot lead to a valid complete solution — even without completing it.
- It uses a depth-first search (DFS) approach on an implicit state-space tree. At each node, it checks a feasibility condition (constraint check). If the check fails, it prunes the subtree and returns to the parent node.
- Difference from brute force: Brute force generates ALL possible candidates and tests each — O(n!) or O(2^n) without any optimization. Backtracking eliminates entire subtrees as soon as a constraint is violated, drastically reducing the number of candidates examined.
- Example: For 8-Queens, brute force places queens in all 8^8 = 16,777,216 arrangements and tests each. Backtracking places queens row by row, pruning as

soon as a conflict is detected — typically examining fewer than 2,000 partial configurations.

Q34 (Basic). Apply backtracking approach to write an algorithm and diagonal logic to solve the 8-Queens problem.

8-Queens Problem

- Place 8 queens on an 8x8 chessboard such that no two queens attack each other (no two in same row, column, or diagonal).

Algorithm

```
function solveQueens(row):
  if row == N:
    print solution; return
  for col = 0 to N-1:
    if isSafe(row, col):
      board[row] = col          // place queen at (row, col)
      solveQueens(row + 1)      // recurse to next row
      board[row] = -1           // backtrack: remove queen

function isSafe(row, col):
  for i = 0 to row-1:
    if board[i] == col:                  return false  // same column
    if abs(board[i]-col) == abs(i-row): return false  // diagonal
  return true
```

Diagonal Logic

- Two queens at (r1,c1) and (r2,c2) are on the same diagonal if: |r1-r2| == |c1-c2|.
- Since we place one queen per row, we only need to check: same column AND same diagonal (both main and anti-diagonal) against all previously placed queens.

Complexity

- Time: O(n!) in worst case; with pruning, effectively O(n^n / n!) which is much less.
- Space: O(n) for the board array and recursion stack.
- For n=8: 92 total solutions exist. Backtracking finds all of them.

Q35 (Basic). Discuss the role of recursion in backtracking algorithms. Why is recursion essential in implementing backtracking strategies?

- Recursion naturally models the state-space tree. Each recursive call represents exploring one branch (one decision at the current level). Returning from a recursive call is the "backtracking" step — it restores the state to before the decision was made.
- Recursion provides an implicit stack: the call stack automatically remembers which decisions were made at each level, eliminating the need for an explicit stack data structure.
- The base case of the recursion corresponds to a complete solution (leaf node). Each recursive call corresponds to branching (extending the partial solution by one decision).
- Undoing a decision: After a recursive call returns (either the branch was exhausted or a solution was found), the state is restored to before the decision. This is the backtracking action — naturally implemented as code after the recursive call.

- Alternative: Iterative backtracking using an explicit stack is possible but significantly more complex to implement. Recursion provides clarity and correctness with minimal code.

Q36 (Basic). Differentiate between backtracking and dynamic programming in terms of problem-solving approach. Provide an example where backtracking would be more appropriate.

Criterion	Backtracking	Dynamic Programming
Approach	DFS with pruning; explores and abandons invalid paths	Bottom-up or top-down with memoization; reuses subproblem results
Subproblems	No explicit reuse — may recompute	Overlapping subproblems stored in table
Solution space	Finds ALL valid solutions or the optimal one	Finds the optimal value/solution
Memory	O(depth) for recursion stack	O(table size) — can be large
Best for	Constraint satisfaction, combinatorial enumeration	Optimization with overlapping subproblems
Example	8-Queens, Graph Coloring, N-Queens	0/1 Knapsack, LCS, Floyd-Warshall

- Backtracking is more appropriate for: Graph Coloring — where we must find a valid coloring (constraint satisfaction) rather than minimize/maximize a value. DP does not apply because graph coloring has no optimal substructure in the DP sense.

Q37 (Basic). Summarize the process of backtracking in solving the Hamiltonian Path problem.

- Hamiltonian Path: Find a path in a graph that visits every vertex exactly once.
- Backtracking approach: Start from a source vertex. At each step, extend the current path to an adjacent unvisited vertex. If the path includes all n vertices, a Hamiltonian Path is found.
- Backtrack when: No unvisited adjacent vertex is available and the path is incomplete. Remove the last vertex from the path and try the next alternative.

```
function hamPath(path, visited):
  if len(path) == n: return true (Hamiltonian Path found)
  for each neighbor v of path.last():
    if not visited[v]:
      add v to path; mark visited[v]=true
      if hamPath(path, visited): return true
      remove v from path; mark visited[v]=false  // backtrack
  return false
```

- Correctness: All paths are explored systematically. Backtracking ensures no vertex is revisited.
- Complexity: O(n!) worst case (similar to brute force for dense graphs). With pruning (dead-end detection), significantly better in practice.

Q38 (Basic). Explain why backtracking is often referred to as a depth-first search (DFS) approach.

- Backtracking explores the state-space tree using DFS: it always extends the current partial solution one step deeper before trying alternatives at the current level.
- Like DFS, backtracking uses a stack (implicit via recursion) to track the current path. When a dead-end is reached (no valid extension), it "pops" the stack by returning from the recursive call — exactly like DFS backtracking to explore another branch.
- DFS visits nodes in pre-order: root -> leftmost subtree -> next subtree. Backtracking does the same: try first choice -> recurse -> if fail, try next choice.
- The key difference: DFS on an explicit graph visits all nodes. Backtracking on an implicit tree prunes subtrees where the constraint check fails — a major efficiency gain over pure DFS.

SECTION 5 — BACKTRACKING : NUMERICAL PROBLEMS

Q39 (Moderate). Demonstrate how the backtracking algorithm is used to place 8 queens on a chessboard such that no two queens threaten each other. Show for a 4x4 chessboard.

Problem Understanding

Place 4 queens on a 4x4 board, one per row, such that no two share a column or diagonal.

Board Representation: board[row] = column of queen in that row.

Step-by-step Dry Run

Row 0: Try col=0. isSafe(0,0)=true. board=[0,-,-,-]
Row 1: Try col=0. Same col as row0. Fail.
Try col=1. Diagonal: |0-1|=1=|0-1|. Fail.
Try col=2. isSafe. board=[0,2,-,-]
Row 2: Try col=0. Diagonal with row1(col2): |2-0|=2=|1-2|? No. Col with row0? Yes (col=0). Fail.
Try col=1. Diagonal with row0: |0-1|=1=|0-2|? No. Diagonal with row1: |2-1|=1=|1-2|? Yes. Fail.
Try col=2. Same col as row1. Fail.
Try col=3. Check row0(col0): |0-3|=3!=|0-2|=2. OK. Check row1(col2): |2-3|=1=|1-2|=1. Fail.
ALL cols fail at row 2. BACKTRACK to row 1.
Row 1: Try col=3. Check row0(col0): |0-3|=3!=|0-1|=1. OK. board=[0,3,-,-]
Row 2: Try col=0. Col=0 same as row0. Fail.
Try col=1. Check row0(col0):ok. Check row1(col3):|3-1|=2=|1-2|=1? No. board=[0,3,1,-]
Row 3: Try col=0. Same col as row0. Fail.
Try col=1. Same col as row2. Fail.
Try col=2. Check row0(0): ok. Check row1(3):|3-2|=1=|1-3|=2? No. Check row2(1):|1-2|=1=|2-3|=1. Fail (diagonal).
Try col=3. Same col as row1. Fail.
BACKTRACK to row 2.

Row 2: Try col=2. Same col as row1(col3)? No. Check row0(col0):ok. Check row1(col3):|3-2|=1=|1-2|=1. Fail.
Try col=3. Same col as row1. Fail.
BACKTRACK to row 1.
ALL cols exhausted for row 1. BACKTRACK to row 0.

Row 0: Try col=1. board=[1,-,-,-]
Row 1: Try col=0. Check(0,1)->|1-0|=1=|0-1|? Yes. Fail.
Try col=1. Same col. Fail.
Try col=2. Check(0,1)->|1-2|=1=|0-1|=1. Fail.
Try col=3. Check(0,1)->|1-3|=2!=|0-1|=1. OK. board=[1,3,-,-]
Row 2: Try col=0. Check row0(1):|1-0|=1=|0-2|=2? No. Check row1(3):|3-0|=3!=|1-2|=1. OK. board=[1,3,0,-]
Row 3: Try col=0. Same col as row2. Fail.
Try col=1. Same col as row0. Fail.
Try col=2. Check row0(1):|1-2|=1=|0-3|=3? No. Check row1(3):|3-2|=1=|1-3|=2? No. Check row2(0):|0-2|=2=|2-3|=1? No. SAFE!
board=[1,3,0,2]. ALL 4 queens placed!

Solution Found: board = [1, 3, 0, 2]

Translation to chessboard coordinates (0-indexed):

Row 0: Queen at column 1 (Q in position (0,1))
Row 1: Queen at column 3 (Q in position (1,3))
Row 2: Queen at column 0 (Q in position (2,0))
Row 3: Queen at column 2 (Q in position (3,2))

```
Col:  0  1  2  3
Row0: .  Q  .  .
Row1: .  .  .  Q
Row2: Q  .  .  .
Row3: .  .  Q  .
```

Valid 4-Queens solution: No two queens share row, column, or diagonal.

Q40 (Moderate). Using backtracking, show how you would assign colors to the vertices of a graph such that no two adjacent vertices share the same color. Apply to a graph with 5 vertices and 3 colors.

Problem Understanding

Graph G: 5 vertices (1-5). Edges: (1-2),(1-3),(2-3),(2-4),(3-5),(4-5). Colors: {1=Red, 2=Green, 3=Blue}.

Algorithm

```
function colorGraph(vertex v, color c[]):
  if v > n: print solution; return
  for color = 1 to m:
    if isSafe(v, color, c):
      c[v] = color
      colorGraph(v+1, c)
      c[v] = 0  // backtrack

isSafe(v, color, c): for each neighbor u of v: if c[u]==color return false;
return true
```

Step-by-step Dry Run

c = [0,0,0,0,0,0] (1-indexed, 0=uncolored)

v=1: Try color=1(Red). No neighbors colored. Safe. c[1]=1.
v=2: Neighbors={1}. c[1]=1=Red. Try color=1: conflict. Try color=2(Green). Safe. c[2]=2.
v=3: Neighbors={1,2}. c[1]=1,c[2]=2. Try 1: conflict(1). Try 2: conflict(2). Try 3(Blue). Safe. c[3]=3.
v=4: Neighbors={2}. c[2]=2=Green. Try 1(Red). Safe. c[4]=1.
v=5: Neighbors={3,4}. c[3]=3=Blue, c[4]=1=Red. Try 1: conflict(c[4]=1). Try 2(Green). Safe. c[5]=2.
v=6 > n=5: Solution found!

Coloring: V1=Red, V2=Green, V3=Blue, V4=Red, V5=Green. (3 colors, no adjacent same color)

Q41 (Moderate). Given a set of numbers and a target sum, demonstrate how backtracking can be used to find all subsets that sum to the target. Use the set {1, 3, 9, 2, 7} and target sum of 10.

Problem Understanding

Find all subsets of {1, 3, 9, 2, 7} that sum exactly to 10.

Algorithm / Approach

```
function subsetSum(index, current_sum, subset):
  if current_sum == target: print subset; return
  if current_sum > target OR index >= n: return (prune)
  // Include A[index]
  subset.add(A[index])
  subsetSum(index+1, current_sum + A[index], subset)
  subset.remove(A[index])   // backtrack
  // Exclude A[index]
  subsetSum(index+1, current_sum, subset)
```

Step-by-step Execution — Key Paths

A = [1,3,9,2,7]. Target = 10.

Path 1: Include 1. Sum=1.
  Include 3. Sum=4.
    Include 9. Sum=13>10. PRUNE.
    Exclude 9. Include 2. Sum=6.
      Include 7. Sum=13>10. PRUNE.
      Exclude 7. Sum=6 != 10. End.
    Exclude 2. Include 7. Sum=11>10. PRUNE.
  Exclude 3. Include 9. Sum=10 == target. SOLUTION: {1,9}
  Exclude 3, 9. Include 2. Sum=3. Include 7. Sum=10. SOLUTION: {1,2,7}

Path 2: Exclude 1. Include 3. Sum=3.
  Include 9. Sum=12>10. PRUNE.
  Exclude 9. Include 2. Sum=5. Include 7. Sum=12>10. PRUNE.
  Exclude 9,2. Include 7. Sum=10. SOLUTION: {3,7}

Path 3: Exclude 1,3. Include 9. Sum=9. Include 2. Sum=11>10. PRUNE. Exclude 2. Include 7. Sum=16>10. PRUNE.
Exclude 1,3,9. Include 2. Sum=2. Include 7. Sum=9!=10. End.
Exclude 1,3,9,2. Include 7. Sum=7!=10. End.

Subsets summing to 10: {1, 9}, {1, 2, 7}, {3, 7}

Q42 (Moderate). Apply the backtracking method to solve a 0/1 Knapsack problem using a space tree. Weights: (5, 3, 4, 2), Profits: (20, 15, 12, 10), Capacity M = 7.

Problem Understanding

- n=4 items. Capacity M=7. Maximize profit subject to weight <= 7. Each item is either included (1) or excluded (0).
- Sorted by P/W ratio: Item2(15/3=5.0), Item4(10/2=5.0), Item1(20/5=4.0), Item3(12/4=3.0).
- Upper Bound at each node: current_profit + fractional fill of remaining items.

Step-by-step Space Tree Traversal (DFS / Backtracking)

Node 0 (root): w=0, p=0. UB = 15+10+(2/5)*20 = 15+10+8 = 33.

Include Item2 (w=3,p=15): w=3,p=15. UB=15+10+(1/5)*20=15+10+4=29.
Include Item4 (w=2,p=10): w=5,p=25. UB=25+(2/5)*20=25+8=33.
Include Item1 (w=5): w=5+5=10>7. INFEASIBLE. Backtrack.
Exclude Item1: w=5,p=25. Include Item3(w=4): 5+4=9>7. INFEASIBLE.
Exclude Item3: p=25,w=5. LEAF. bestProfit=25. Solution:{Item2,Item4}.
Exclude Item4: w=3,p=15. UB=15+(4/5)*20=15+16=31.
Include Item1 (w=5): w=3+5=8>7. INFEASIBLE.
Exclude Item1: w=3,p=15. Include Item3(w=4):3+4=7<=7. p=15+12=27. LEAF. bestProfit=27. Solution:{Item2,Item3}.
Exclude Item3: p=15. LEAF. Less than 27.

Exclude Item2: w=0,p=0. UB=10+(4/5)*20+(3/4)*12=10+16+9=35.
Include Item4(w=2,p=10): w=2,p=10. UB=10+(4/5)*20+(3/4)*12=35.
Include Item1(w=5): w=7,p=30. LEAF. bestProfit=30. Solution:{Item4,Item1}.
Exclude Item1: w=2,p=10. Include Item3(w=4): w=6,p=22. LEAF. p=22<30.
Exclude Item4: w=0,p=0. UB=20+12=32. (After Item4 excluded, consider Item1,Item3)
Include Item1(w=5,p=20): LEAF path with Item1+Item3: w=9>7. Only Item1: w=5,p=20<30.

Maximum Profit = 30. Selected items: Item4 (w=2,p=10) + Item1 (w=5,p=20). Total weight = 7.

Q43 (Moderate). Apply backtracking method to get maximum profit using 0/1 Knapsack space tree. Weights: (2, 1, 3, 2), Profits: (12, 10, 20, 15), Capacity M = 5.

Problem Understanding

- n=4 items. Capacity M=5.

Item	Weight	Profit	P/W

1	2	12	6.0
2	1	10	10.0
3	3	20	6.67
4	2	15	7.5

- Sort by P/W ratio: Item2(10), Item4(7.5), Item3(6.67), Item1(6.0).
- Reorder: Item2(w=1,p=10), Item4(w=2,p=15), Item3(w=3,p=20), Item1(w=2,p=12).

Step-by-step Space Tree

Root: w=0,p=0. UB=10+15+(1/3)*20+... = 10+15+6.67=31.67.

Include Item2(w=1,p=10): w=1,p=10. UB=10+15+(1/3)*20=31.67.
Include Item4(w=2,p=15): w=3,p=25. UB=25+(2/3)*20=25+13.3=38.3.
Include Item3(w=3): w=6>5. INFEASIBLE.
Exclude Item3: w=3,p=25. Include Item1(w=2): w=5,p=37. LEAF. bestProfit=37. {Item2,Item4,Item1}
Exclude Item1: p=25. LEAF. Less.
Exclude Item4: w=1,p=10. UB=10+20+(2/3)*12=10+20+8=38.
Include Item3(w=3): w=4,p=30. UB=30+(1/2)*12=36.
Include Item1(w=2): w=6>5. INFEASIBLE.
Exclude Item1: p=30. LEAF. p=30<37.
Exclude Item3: w=1,p=10. Include Item1(w=2): w=3,p=22. LEAF. p=22<37.

Exclude Item2: w=0,p=0. UB=15+20+(2/2)*12=15+20+12=47? Weight check:
Include Item4(w=2,p=15): w=2. Include Item3(w=3): w=5,p=35. LEAF. p=35<37.
Include Item4+Item1(w=4,p=27). LEAF. p=27<37.
All paths < 37. No improvement.

Maximum Profit = 37. Selected items: Item2 (w=1,p=10) + Item4 (w=2,p=15) + Item1 (w=2,p=12). Total weight = 5.

SECTION 6 — BACKTRACKING : HOTS QUESTIONS

Q44 (HOTS). Analyze the time complexity of the backtracking algorithm used to solve the 8-Queens problem. How does the search space affect the overall performance?

- Brute force search space: Place a queen in each of 8 rows choosing from 8 columns -> 8^8 = 16,777,216 configurations. Backtracking checks validity row by row.
- Since we place exactly one queen per row, the actual candidates per row decrease as queens are placed. The upper bound on the search space is 8! = 40,320 (permutations of columns).
- With diagonal pruning, many column assignments are rejected early. In practice, backtracking explores approximately 15,720 nodes for the 8-Queens problem — fewer than 0.1% of brute force configurations.
- General n-Queens: Time complexity is O(n!) in the worst case (no pruning), but with constraint propagation (arc consistency), it can be reduced to approximately O(n^1.5 * something) — significantly less than n!.

- Search space effect: Denser constraint graphs (more mutual attacks) lead to more aggressive pruning. For 8-Queens on an 8x8 board, pruning is highly effective because diagonal constraints eliminate most partial solutions early.
- Number of solutions: 8-Queens has exactly 92 solutions (12 fundamental, 80 reflections/rotations). For n=14, there are 365,596 solutions; for n=20, over 2 billion — showing exponential growth in solutions and search space.

Q45 (HOTS). Compare the backtracking approach for the 8-Queens problem with other potential approaches (e.g., brute force). Which approach is more efficient, and why?

Approach	Search Space	Pruning	Efficiency	For n=8
Brute Force (all arrangements)	n^n = 8^8 = 16.7M	None	Very poor	16.7M checks
Brute Force (permutations)	n! = 40,320	Only column conflict	Better	40,320 checks
Backtracking (row-by-row + diagonal prune)	<< n!	Column + diagonal	Excellent	~15,720 nodes
Constraint Propagation (FC/AC-3)	Much less than BT	Forward checking removes future domains	Best	< 200 nodes

- Backtracking with diagonal constraint checking is 1000x more efficient than basic brute force for n=8. The efficiency gap grows exponentially with n.
- Constraint Propagation (Forward Checking): After placing a queen, immediately remove attacked cells from the domains of all future queens. This reduces backtracking significantly — an even more advanced form of backtracking.

Q46 (HOTS). Analyze the efficiency of the backtracking algorithm for solving the graph coloring problem. What factors influence performance?

- Graph coloring with backtracking: Assign colors vertex by vertex. At each vertex, try all m colors; backtrack if a color causes a conflict with an adjacent already-colored vertex.
- Worst-case complexity: O(m^n) — try all m colors for each of n vertices. With pruning (reject a color immediately if it conflicts), this is drastically reduced.

Factors that influence performance:

- 1. Graph density (number of edges): Denser graphs have more constraints, causing more pruning — paradoxically, very dense graphs may be solved faster because conflicts are detected earlier.
- 2. Chromatic number X(G): If m >> X(G), many color assignments work and the algorithm finds solutions quickly. If m is close to X(G), the problem is harder.
- 3. Variable ordering heuristic: Applying the "Most Constrained Variable" (MCV) heuristic — color the vertex with the most colored neighbors first — reduces backtracking significantly.
- 4. Value ordering heuristic: Try the "Least Constraining Value" first — the color that eliminates the fewest options for remaining vertices. Reduces backtracking.

- 5. Constraint propagation: After coloring a vertex, remove that color from the domains of all adjacent vertices. Detects failures earlier.
- Practical performance: For planar graphs (X(G) <= 4 by Four Color Theorem), backtracking with m=4 finds a valid coloring very efficiently. For general graphs, performance degrades with graph density and m close to X(G).

Q47 (HOTS). Compare the backtracking approach for graph coloring with greedy algorithms and dynamic programming.

Criterion	Backtracking	Greedy	Dynamic Programming
Optimality (min colors)	YES — finds chromatic number	NO — may use more colors than needed	Not directly applicable for graph coloring
All solutions	YES — finds all valid colorings	No — gives one (possibly suboptimal) coloring	N/A
Time complexity	O(m^n) worst case with pruning	O(n + E) — linear	N/A for general graph coloring
Space	O(n) recursion stack	O(n)	N/A
Practical use	Small graphs, finding chromatic number	Large graphs where a near-optimal coloring suffices	N/A

- DP is not directly applicable to graph coloring because the problem does not have the optimal substructure property in the general case — the coloring of one subset of vertices depends on the global structure of the graph.
- Greedy coloring with DSatur (degree of saturation) heuristic gives near-optimal results in practice with O(n^2) complexity — a good compromise between backtracking (optimal but slow) and simple greedy (fast but potentially many extra colors